

Cubical Agda: A dependently typed programming language with univalence and higher inductive types

ANDREA VEZZOSI 

Department of Computer Science, IT University of Copenhagen, Copenhagen, Denmark
(e-mail: avez@itu.dk)

ANDERS MÖRTBERG

Department of Mathematics, Stockholm University, Stockholm, Sweden
(e-mail: anders.mortberg@math.su.se)

ANDREAS ABEL

Department of Computer Science and Engineering, Chalmers and Gothenburg University, Gothenburg, Sweden
(e-mail: andreas.abel@gu.se)

Abstract

Proof assistants based on dependent type theory provide expressive languages for both programming and proving within the same system. However, all of the major implementations lack powerful extensionality principles for reasoning about equality, such as function and propositional extensionality. These principles are typically added axiomatically which disrupts the constructive properties of these systems. Cubical type theory provides a solution by giving computational meaning to Homotopy Type Theory and Univalent Foundations, in particular to the univalence axiom and higher inductive types (HITs). This paper describes an extension of the dependently typed functional programming language Agda with cubical primitives, making it into a full-blown proof assistant with native support for univalence and a general schema of HITs. These new primitives allow the direct definition of function and propositional extensionality as well as quotient types, all with computational content. Additionally, thanks also to copatterns, bisimilarity is equivalent to equality for coinductive types. The adoption of cubical type theory extends Agda with support for a wide range of extensionality principles, without sacrificing type checking and constructivity.

1 Introduction

A core idea in programming and mathematics is *abstraction*: the exact details of how an object is represented should not affect its abstract properties. In other words, the implementation details should not matter. This is exactly what the principle of univalence captures by extending the equality on the universe of types to incorporate equivalent types.¹ This then

¹ For the sake of this introduction, “equivalent” may be read as “isomorphic.” In Homotopy Type Theory (HoTT), *isomorphism* coincides with equivalence for *sets* (in the sense of HoTT). Equivalence for *types* in general is a refinement of the concept of isomorphism.

gives a form of abstraction, or invariance up to equivalence, in the sense that equivalent types will share the same structures and properties. The fact that equality is *proof relevant* in dependent type theory is the key to enabling this; the data of an equality proof can store the equivalence, and transporting along this equality should then apply the function underlying the equivalence. In particular, this allows programs and properties to be transported between equivalent types, hereby increasing modularity and decreasing code duplication. A concrete example are the equivalent representations of natural numbers in unary and binary format. In a univalent system, it is possible to develop the theory of natural numbers using the unary representation but compute using the binary representation, and as the two representations are equivalent they share the same properties.

The principle of univalence is the major new addition in Homotopy Type Theory and Univalent Foundations (HoTT/UF) ([Univalent Foundations Program, 2013](#)). However, these new type-theoretic foundations add univalence as an *axiom* which disrupts the good constructive properties of type theory. In particular, if we transport addition on binary numbers to the unary representation, we will not be able to compute with it as the system would not know how to reduce the univalence axiom. Cubical type theory ([Cohen et al., 2018](#)) addresses this problem by introducing a novel representation of equality proofs and thereby providing computational content to univalence. This makes it possible to constructively transport programs and properties between equivalent types. This representation of equality proofs has many other useful consequences, in particular function and propositional extensionality (pointwise equal functions and logically equivalent propositions are equal), and the equivalence between bisimilarity and equality for coinductive types ([Vezzosi, 2017](#)).

Dependently typed functional languages such as Agda ([2018](#)), Coq ([2019](#)), Idris ([Brady, 2013](#)), and Lean ([de Moura et al., 2015](#)) provide rich and expressive environments supporting both programming and proving within the same language. However, the extensionality principles mentioned above are not available out of the box and need to be assumed as axioms just as in HoTT/UF. Unsurprisingly, this suffers from the same drawbacks as it compromises the computational behavior of programs that use these axioms, and even make subsequent proofs more complicated.

So far, cubical type theory has been developed with the help of a prototype Haskell implementation called `cubicaltt` ([Cohen et al., 2015](#)), but it has not been integrated into one of the main dependently typed functional languages. Recently, an effort was made, using Coq, to obtain effective transport for restricted uses of the univalence axiom ([Tabareau et al., 2018](#)), because, as the authors mention, “*it is not yet clear how to extend [proof assistants] to handle univalence internally.*”

This paper achieves this, and more, by making Agda into a cubical programming language with native support for univalence and higher inductive types (HITs). We call this extension Cubical Agda as it incorporates and extends cubical type theory. In addition to providing a fully constructive univalence theorem, Cubical Agda extends the theory by allowing proofs of equality by copatterns, HITs as in Coquand et al. ([2018](#)) with nested pattern matching, and interval and partial pretypes. This paper aims to provide a formal account of the extensions to the language of Agda and its type-checking algorithm needed to accommodate the new features. In particular, as it requires the most care, we will dedicate a large portion of this paper to the handling of pattern matching.

Contributions. The main contribution of this paper is the implementation of Cubical Agda, a fully fledged proof assistant with constructive support for univalence and HITs. This makes a variety of extensionality principles provable and we show how these can be used for programming and proving in [Section 2](#). We explain how Agda was extended to support cubical type theory ([Section 3](#)); in particular, we describe how some primitive notions of cubical type theory are internalized as pretypes: the interval ([Section 3.1](#)), partial elements, and cubical subtypes ([Section 3.3](#)). The technical contributions are

- We extend cubical type theory by records and coinductive types ([Section 3.2.2](#)).
- We add support for a general schema of HITs and extend the powerful dependent pattern matching of Agda to also support pattern matching on HITs ([Section 4](#)).
- We include support for inductive families, which also requires extra care to handle pattern-matching definitions ([Section 4](#)).
- We describe an optimization to the algorithm for transport in Glue types ([Section 5](#)), which gives a simpler proof of the univalence theorem compared to Cohen *et al.* (2018) ([Section 5.2](#)).

Using the optimization to transport for Glue types, we discuss an improved canonicity theorem for cubical type theory with HITs ([Section 6](#)). The paper finishes with some concluding remarks and an overview of related and future work ([Section 7](#)).

A conference version of this article has appeared at the *International Conference on Functional Programming* (Vezzosi *et al.*, 2019). The support for inductive *families* (rather than just inductive *types*) and the related examples are the novel contributions of the present journal version of this article. At the time of writing, the implementation of inductive families has not yet been merged into the main branch of Agda, but it is available through <https://github.com/agda/agda/tree/issue3733>. The required extension to the proof-relevant unifier (see [Section 4.3.1](#)) does not yet handle unification by injectivity of constructors.

2 Programming and proving in Cubical Agda

In this section, we show some examples of how the new cubical features in Agda enable interesting and useful ways for both programming and proving in dependent type theory. No expert knowledge of HoTT/UF is assumed. Using univalence and other ideas from HoTT/UF, we can

1. Transfer programs and proofs between equivalent types ([Section 2.1.1](#));
2. Prove properties for proof-oriented datatypes using computation-oriented ones ([Section 2.1.2](#));
3. Reason about dependently typed programs using inductive families ([Section 2.2](#));
4. Treat bisimilar elements of coinductive types as equal ([Section 2.3](#));
5. Define and reason about quotient types ([Section 2.4](#));
6. Represent topological spaces as datatypes and reason about them synthetically ([Section 2.5](#)).

The examples are taken from the open-source library `agda/cubical` hosted at <https://github.com/agda/cubical>.

2.1 Unary and binary numbers

An example of two equivalent types that are well suited for different tasks are unary and binary numbers. The unary representation is useful for proving because of its direct induction principle and the binary representation is much better for computation as it is exponentially more compact. By utilizing computational univalence, we can transfer results between the two representations in a convenient way; this gives us the best of both worlds without having to duplicate results.

The unary numbers, $\mathbb{N}$, are built into Agda and are inductively generated by the constructors `zero` and `suc` (successor). We encode binary numbers as:

```
data Bin : Set where
  bin0   : Bin
  binPos : Pos → Bin
```

```
data Pos : Set where
  pos1 : Pos
  x0   : Pos → Pos
  x1   : Pos → Pos
```

A binary number is hence either zero (`bin0`) or a positive binary number represented as a list of zeroes and ones with no trailing zeroes. Least significant bits come first (little-endian format), thus, the number 6 is binary 011 (`binPos (x0 (x1 pos1)))`). This way, every number has a unique binary representation, and it is straightforward to write maps to and from the unary representation ($\text{Bin} \rightarrow \mathbb{N}$ and $\mathbb{N} \rightarrow \text{Bin}$) with proofs that they cancel ($\mathbb{N} \rightarrow \text{Bin} \rightarrow \mathbb{N}$ and $\text{Bin} \rightarrow \mathbb{N} \rightarrow \text{Bin}$). This means that the two types $\mathbb{N}$ and Bin are isomorphic which implies that they are *equivalent*, in the sense of the terminology of Voevodsky (2015) and Univalent Foundations Program (2013). Spelled out, a map $f : A \rightarrow B$ is an equivalence if the preimage of any point in B is a singleton type.

Given types A and B , we write $A \simeq B$ for the type of equivalences between them, and the univalence theorem² then states that

$$(A \equiv B) \simeq (A \simeq B).$$

In particular, there is a function $\text{ua} : A \simeq B \rightarrow A \equiv B$, sending a proof that two types are equivalent to an equality between these types. We use ua to turn the equivalence of $\mathbb{N}$ and Bin into an equality:

```
 $\mathbb{N} \simeq \text{Bin} : \mathbb{N} \simeq \text{Bin}$ 
 $\mathbb{N} \simeq \text{Bin} = \text{isoToEquiv } (\text{iso } \mathbb{N} \rightarrow \text{Bin } \text{Bin} \rightarrow \mathbb{N} \text{ Bin} \rightarrow \mathbb{N} \rightarrow \text{Bin} \rightarrow \mathbb{N})$ 
 $\mathbb{N} \equiv \text{Bin} : \mathbb{N} \equiv \text{Bin}$ 
 $\mathbb{N} \equiv \text{Bin} = \text{ua } \mathbb{N} \simeq \text{Bin}$ 
```

² As the univalence “axiom” is provable in Cubical Agda, we refer to it as the *univalence theorem*.

In fact, the equality in $\mathbb{N} \equiv \text{Bin}$ is not the regular type-theoretic equality à la Martin-Löf (in the sense of being inductively generated from constructor `refl` for reflexivity), but rather a *path* equality. The core idea of HoTT/UF is the close correspondence between proof-relevant equality, as in type theory, and paths, as in topology. The idea that equality corresponds to paths is taken very literally in cubical type theory; by adding a primitive interval type `I`, paths in a type A can be represented as functions $I \rightarrow A$. Iterating these function types lets us represent squares, cubes, and hypercubes, making the type theory *cubical*.

The interval `I` has two distinguished endpoints `i0` and `i1`. Since paths are functions, we introduce them using λ -abstraction and eliminate them using function application; by applying a path to `i0`, we get its left endpoint and by applying it to `i1` we get the right one. We often want to specify the endpoints of a path (or, more generally, the boundary of a cube) in its type; in Cubical Agda, there is a special primitive for this:

`PathP` : $(A : I \rightarrow \text{Set } \ell) \rightarrow A \text{ i0} \rightarrow A \text{ i1} \rightarrow \text{Set } \ell$

we introduce these paths by lambda abstractions like so, $\lambda i \rightarrow t$: `PathP A (t[i0/i]) (t[i1/i])`, provided that $t : A\ i$ for $i : I$. Consequently, we can apply $p : \text{PathP } A\ a_0\ a_1$ to an $r : I$ to obtain $p\ r : A\ r$. Also, no matter how p is given, we have that $p\ \text{i0}$ reduces to a_0 and $p\ \text{i1}$ reduces to a_1 .

The `PathP` types should be thought of as heterogeneous equalities, since the two endpoints are in different types; this is similar to the dependent paths in HoTT (Univalent Foundations Program, 2013, Section 6.2). We can define homogeneous nondependent path equality in terms of `PathP` as follows:

`_≡_` : $\{A : \text{Set } \ell\} \rightarrow A \rightarrow A \rightarrow \text{Set } \ell$
`_≡_` $\{A = A\} x\ y = \text{PathP } (\lambda _ \rightarrow A) x\ y$

In the previous definition, the syntax $\{A = A\}$ tells Agda to bind the hidden argument A of `_≡_` (the first A in $\{A = A\}$) to a variable A (the second A) that can be used on the right-hand side. Note also that some definitions are polymorphic in universe level ℓ which is implicitly universally quantified. Further, from now, we will omit the explicit quantification over $\{A : \text{Set } \ell\}$.

Viewing equalities as functions out of the interval allows us to reason elegantly about equality; for instance, the constant path represents a proof of reflexivity:

`refl` : $\{x : A\} \rightarrow x \equiv x$
`refl` $\{x = x\} = \lambda i \rightarrow x$

We can also directly apply a function to a path in order to prove that dependent functions respect path equality, as shown in the definition of `cong` below. Simply by computation, `cong` satisfies some new definitional equalities compared to the corresponding definition for the inductive equality type à la Martin-Löf (1975). For instance, `cong` is functorial by definition: we can prove `conglid` and `congComp` by plain reflexivity (`refl`):

`cong` : $\forall \{B : A \rightarrow \text{Set } \ell\} (f : (a : A) \rightarrow B\ a) \{x\ y\} (p : x \equiv y) \rightarrow$
 $\text{PathP } (\lambda i \rightarrow B\ (p\ i)) (f\ x) (f\ y)$
`cong` $f\ p\ i = f\ (p\ i)$
`conglid` : $\forall \{x\ y : A\} (p : x \equiv y) \rightarrow \text{cong } (\lambda a \rightarrow a) p \equiv p$

`conglD` $p = \text{refl}$

`congComp` : $\forall (f : A \rightarrow B) (g : B \rightarrow C) \{x\ y\} (p : x \equiv y) \rightarrow$
 $\text{cong } (g \circ f) p \equiv \text{cong } g (\text{cong } f p)$

`congComp` $f g p = \text{refl}$

Path types also let us prove new things that are not provable in standard Agda with Martin-Löf propositional equality. For example, function extensionality, stating that pointwise equal functions are equal themselves, has an extremely simple proof:

`funExt` : $\{f\ g : A \rightarrow B\} \rightarrow ((x : A) \rightarrow f x \equiv g x) \rightarrow f \equiv g$

`funExt` $p\ i\ x = p\ x\ i$

The proof of function extensionality for dependent and n -ary functions is equally direct. Since `funExt` is a *definable* notion in Cubical Agda, it is, in contrast to Martin-Löf type theory, not an axiom. This means that it has computational content: it simply swaps the arguments to p .

The facts that paths can be manipulated as functions and that we have heterogeneous path types makes many equality proofs simpler compared to the corresponding proofs in standard Agda or HoTT. For instance, the equality of second projections of dependent pair types is a simple instance of `cong`:

$\Sigma\text{-eq}_2 : \forall \{A : \text{Set}\} \{B : A \rightarrow \text{Set}\} \{p\ q : \Sigma [x \in A] (B\ x)\} \rightarrow (e : p \equiv q) \rightarrow$

$\text{PathP } (\lambda i \rightarrow B\ (e\ i.\text{fst}))\ (p.\text{snd})\ (q.\text{snd})$

$\Sigma\text{-eq}_2 = \text{cong } \text{snd}$

The corresponding result in regular type theory has to be stated with the homogeneous notion of equality using transport, making equality in dependent pair types notoriously difficult to work with.

2.1.1 Univalent transport

One of the key properties of type-theoretic equality is *transport*:

`transport` : $A \equiv B \rightarrow A \rightarrow B$

`transport` $p\ a = \text{transp } (\lambda i \rightarrow p\ i)\ i0\ a$

This is defined using another primitive of Cubical Agda called `transp`. It is a generalization of the regular transport principle which lets us specify where the transport is the identity function. In particular, when the second argument to `transp` is `i1`, it will reduce to a , which let us prove that `transp` $A\ r\ a$ is always path equal to a (cf. `addep` later). A consequence is that whenever we have an equivalence $e : A \simeq B$, we have that `transport` $(\lambda i \rightarrow F\ (\text{ua } e\ i))$ is an equivalence as well. This ability to lift equivalences through arbitrary type operators F is an easily overlooked benefit of a language with computational univalence.

The substitution principle is obtained as an instance of `transport`:

`subst` : $(B : A \rightarrow \text{Set } \ell) \{x\ y : A\} \rightarrow x \equiv y \rightarrow B\ x \rightarrow B\ y$

`subst` $B\ p\ b = \text{transport } (\lambda i \rightarrow B\ (p\ i))\ b$

Function `subst` invokes `transport` with a proof of $B\ x \equiv B\ y$; this proof $\lambda i \rightarrow B\ (p\ i)$ is an inlining of `cong`, stating that families B respect equality p .

After this digression about path types, let us revisit the $\mathbb{N} \equiv \text{Bin}$ path. Using `transport`, we can transfer `zero` along $\mathbb{N} \equiv \text{Bin}$, and since univalence is a theorem with computational content in Cubical Agda, this will reduce to `bin0`. In contrast, if we were working in a system with axiomatic univalence, we could still define $\mathbb{N} \equiv \text{Bin}$, but the transport of `zero` along that equality would be *stuck*; the system would not know how to automatically transport with the `ua` constant.

Having computational univalence lets us do a lot more than just transporting constructors. We can for example transport the *addition* function from unary to binary numbers in order to make it easier to prove properties about the more complex binary addition function:

```
_+Bin_ : Bin → Bin → Bin
_+Bin_ = transport (λ i →  $\mathbb{N} \equiv \text{Bin}$  i →  $\mathbb{N} \equiv \text{Bin}$  i →  $\mathbb{N} \equiv \text{Bin}$  i) _+_
```

In this case, the path that we transport along is between the function types $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ and $\text{Bin} \rightarrow \text{Bin} \rightarrow \text{Bin}$. This way, we obtain an addition function on binary numbers and the fact that `ua` has computational content lets us run it:

```
_ : (binPos (x0 (x0 pos1))) +Bin (binPos pos1)  $\equiv$  binPos (x1 (x0 pos1))
_ = refl
```

In order to reduce the left-hand side, Cubical Agda will convert all of the arguments to the unary representation, add them using `_+_`, and then convert the result back to binary. The main reason for defining `_+Bin_` like this is that it lets us transport results about the unary addition function. For example, we transport the proof of associativity as follows:

```
addp : PathP (λ i →  $\mathbb{N} \equiv \text{Bin}$  i →  $\mathbb{N} \equiv \text{Bin}$  i →  $\mathbb{N} \equiv \text{Bin}$  i) _+_ _+Bin_
addp i = transp (λ j →  $\mathbb{N} \equiv \text{Bin}$  (i ∧ j) →  $\mathbb{N} \equiv \text{Bin}$  (i ∧ j) →  $\mathbb{N} \equiv \text{Bin}$  (i ∧ j)) (∼ i) _+_

+Bin-assoc : (m n o : Bin) → m +Bin (n +Bin o)  $\equiv$  (m +Bin n) +Bin o
+Bin-assoc =
  transport (λ i → (m n o :  $\mathbb{N} \equiv \text{Bin}$  i) → addp i m (addp i n o)
             $\equiv$  addp i (addp i m n) o)
  +-assoc
```

In `addp`, we utilize the interval operators *minimum* (`_∧_`) and *reversal* (`∼_`); further, Cubical Agda features the *maximum* operator (`_∨_`). The intuition is that elements of `I` correspond to points in the real unit interval $[0, 1]$. The `_∧_` and `_∨_` operations take the minimum and maximum of $i, j : I$, while the reversal operation computes $1 - i$. These operations satisfy the laws of a *De Morgan algebra*. This means, for one, that the min/max operations form a bounded distributive lattice, with `i0` and `i1` as bottom and top elements. Further, the reversal is a De Morgan involution, so, for instance, $\sim (i \wedge j) = \sim i \vee \sim j$. Note that this is still *not* a Boolean algebra, since $i \wedge \sim i = i0$ and $i \vee \sim i = i1$ are *not* valid for points of the unit interval, except for the endpoints.

Let us assert the well-typedness of `addp`: when i is `i0`, the first argument to `transp` in `addp` is constantly $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$, since `i0 ∧ i` is then just `i0` and $\mathbb{N} \equiv \text{Bin}$ `i0` reduces to $\mathbb{N}$. The second argument `∼ i` becomes `i1`, so that the left endpoint of the path is `_+_`, exploiting that `transp (...)` is the identity function when applied to `i1`. On the other hand, when i is `i1`, then `addp i` reduces to `transp (λ j →  $\mathbb{N} \equiv \text{Bin}$  j →  $\mathbb{N} \equiv \text{Bin}$  j →  $\mathbb{N} \equiv \text{Bin}$  j) i0 _+_` which is exactly the definition of `_+Bin_`. This establishes that `addp` indeed constitutes a

path from `_+_` to `+_Bin_`. Note that the path type of `addp` is heterogeneous as the two addition functions have different types.

The desired result is then obtained by transporting the proof that unary addition is associative along a path from

$$(m\ n\ o : \mathbb{N}) \rightarrow m + (n + o) \equiv (m + n) + o$$

to

$$(m\ n\ o : \text{Bin}) \rightarrow m +_{\text{Bin}} (n +_{\text{Bin}} o) \equiv (m +_{\text{Bin}} n) +_{\text{Bin}} o.$$

The above proof might seem quite complex and the reader might rightfully question the scalability to more involved examples. However, one can largely simplify things using `subst` in a suitable Σ -type:

`T : Set → Set`

`T X =  $\Sigma$ [_+_ ∈ (X → X → X)] ((x y z : X) → x + (y + z) ≡ (x + y) + z)`

`TBin : T Bin`

`TBin = subst T  $\mathbb{N} \equiv_{\text{Bin}}$  (_+_ , +-assoc)`

`_+_Bin'_ : Bin → Bin → Bin`

`_+_Bin'_ = fst TBin`

`+Bin'-assoc : (m n o : Bin) → m +Bin' (n +Bin' o) ≡ (m +Bin' n) +Bin' o`

`+Bin'-assoc = snd TBin`

The operation `_+_Bin'_` is *definitionally* that same as `_+_Bin_`. The user hence does not have to write the proof of `+Bin'-assoc` by hand, but Cubical Agda can compute the addition operation with its associativity proof for them. It is now easy to imagine automatic transport of more complex operations and properties simply by modifying the type family `T`.

As discussed above, the `_+_Bin_` operation is of course a very inefficient way of adding binary numbers. However, we can also define an efficient addition function `_+_B_` as follows:

`mutual`

`_+_P_ : Pos → Pos → Pos`

`pos1 +P y = sucPos y`

`x0 x +P pos1 = x1 x`

`x0 x +P x0 y = x0 (x +P y)`

`x0 x +P x1 y = x1 (x +P y)`

`x1 x +P pos1 = x0 (sucPos x)`

`x1 x +P x0 y = x1 (x +P y)`

`x1 x +P x1 y = x0 (x +PC y)`

`_+_B_ : Bin → Bin → Bin`

`bin0 +B y = y`

`x +B bin0 = x`

`binPos x +B binPos y = binPos (x +P y)`

`- Add with carry`

`_+_PC_ : Pos → Pos → Pos`

`pos1 +PC pos1 = x1 pos1`

`pos1 +PC x0 y = x0 (sucPos y)`

`pos1 +PC x1 y = x1 (sucPos y)`

`x0 x +PC pos1 = x0 (sucPos x)`

`x0 x +PC x0 y = x1 (x +P y)`

`x0 x +PC x1 y = x0 (x +PC y)`

`x1 x +PC pos1 = x1 (sucPos x)`

`x1 x +PC x0 y = x0 (x +PC y)`

`x1 x +PC x1 y = x1 (x +PC y)`

This function is rather complicated as the helper function $_+P_$ for adding positive numbers has to be defined mutually with an addition with carry operation $_+PC_$ in order to be efficient. We do not expect the reader to understand the details, but it should be clear that directly proving properties like associativity for this operation would be very complicated as the deeply nested pattern matching would quickly lead to an explosion of cases. Luckily, we can take advantage of the naive addition operation $_+Bin_$ which we know share all properties of the unary addition $_+_$. The key lemma we need to prove is

$$\mathbb{N} \rightarrow \text{Bin} \rightarrow \text{B} : (x\ y : \mathbb{N}) \rightarrow \mathbb{N} \rightarrow \text{Bin} \ (x + y) \equiv \mathbb{N} \rightarrow \text{Bin} \ x + \text{B} \ \mathbb{N} \rightarrow \text{Bin} \ y$$

As this lemma is expressed by quantification over unary numbers, we largely avoid the explosion of cases. Combining function extensionality with the fact that $\mathbb{N} \rightarrow \text{Bin}$ constitutes one direction of an equivalence, we can easily construct a path $+B \equiv +Bin$ proving that $_+B_$ and $_+Bin_$ are equal functions:

$$+B \equiv +Bin : _+B_ \equiv _+Bin_$$

$$+B \equiv +Bin \ i \ x \ y = \text{goal} \ x \ y \ i$$

where

$$\text{goal} : (x\ y : \text{Bin}) \rightarrow x + \text{B} \ y \equiv \mathbb{N} \rightarrow \text{Bin} \ (\text{Bin} \rightarrow \mathbb{N} \ x + \text{Bin} \rightarrow \mathbb{N} \ y)$$

$$\text{goal} \ x \ y = (\lambda \ i \rightarrow \text{Bin} \rightarrow \mathbb{N} \rightarrow \text{Bin} \ x \ (\sim i) + \text{B} \ \text{Bin} \rightarrow \mathbb{N} \rightarrow \text{Bin} \ y \ (\sim i))$$

- $\text{sym} \ (\mathbb{N} \rightarrow \text{Bin} \rightarrow \text{B} \ (\text{Bin} \rightarrow \mathbb{N} \ x) \ (\text{Bin} \rightarrow \mathbb{N} \ y))$

The $\bullet$ operation is binary composition of paths which will be discussed in detail in Section 3.4. Finally, as the functions are proved equal, they share the same properties; for example, we can turn the proof of $+Bin\text{-assoc}$ into a proof that $_+B_$ is also associative as follows:

$$+B\text{-assoc} : (m\ n\ o : \text{Bin}) \rightarrow m + \text{B} \ (n + \text{B} \ o) \equiv (m + \text{B} \ n) + \text{B} \ o$$

$$+B\text{-assoc} \ m\ n\ o = (\lambda \ i \rightarrow +B \equiv +Bin \ i \ m \ (+B \equiv +Bin \ i \ n \ o))$$

- $+Bin\text{-assoc} \ m\ n\ o$

- $(\lambda \ i \rightarrow +B \equiv +Bin \ (\sim i) \ (+B \equiv +Bin \ (\sim i) \ m \ n) \ o)$

This example shows how univalent transport can be used to reason conveniently about efficient functions on computation-oriented types which would otherwise have been very complicated to do directly. Another useful consequence of being able to transport proofs between types is that we can prove some result by computation for binary numbers and then transport the proof to the unary representation where the computation might have been infeasible.

2.1.2 Univalent program and data refinements

Sometimes, concrete computations are necessary in proofs. For example, one could imagine a situation where one needs to check an equality between two terms that are expensive to compute like:

$$2^{20} \cdot 2^{10} = 2^5 \cdot 2^{15} \cdot 2^{10}$$

When this is part of a proof, it is likely that one is using a unary representation of natural numbers. However, that makes it impossible to verify the above equation by computation. One way to resolve this dilemma would be to redo the formalization using binary numbers,

but that could involve a complete rewrite of the formalization. Another alternative would be to use algebraic manipulations to prove the above equality manually. However, the latter is sometimes not feasible as the computation might be very complicated.

Such issues can be resolved by what we call *univalent program and data refinements* following Cohen *et al.* (2013). As binary numbers are equivalent to unary numbers, we can prove the property for binary numbers by computation and then transport the proof to unary numbers. To this end, we define a “doubling structure” in which we can express the above equation and instantiate with unary and binary numbers. We omit the concrete definitions, but as expected the doubling function is of linear complexity for unary numbers and constant for binary:

```
record Double (A : Set) : Set where
  constructor doubleStruct
  field
    double : A → A
    elt : A
```

```
DoubleN : Double N
DoubleN = doubleStruct doubleN 1024
```

```
DoubleBin : Double Bin
DoubleBin = doubleStruct doubleBin bin1024
```

The equality between binary and unary numbers lifts to an equality of doubling structures. We omit the details of this definition as they are quite technical; however, the whole definition is only 7 lines of code so it is not particularly difficult to write once one is familiar with all of the features of Cubical Agda :

```
DoubleBin≡DoubleN : PathP (λ i → Double (Bin≡N i)) DoubleBin DoubleN
```

We can now formulate the equation that we originally wanted to verify. We wrap the equation in a record and use copattern matching when proving it for the `Bin` instance (`DoubleBin`). This technical trick prevents Agda from eagerly unfolding the `N` instance when we transport the proof over to unary numbers along the equality of doubling structures. Section 3.2.2 discusses transport in record types:

```
doubles : {A : Set} (D : Double A) → N → A → A
doubles D n x = iter n (double D) x
```

```
record propDouble {A : Set} (D : Double A) : Set where
  field
    proof : doubles D 20 (elt D) ≡ doubles D 5 (doubles D 15 (elt D))
```

```
propDoubleBin : propDouble DoubleBin
proof propDoubleBin = refl
```

```
propDoubleN : propDouble DoubleN
propDoubleN = transport (λ i → propDouble (DoubleBin≡DoubleN i))
  propDoubleBin
```

The fact that equivalences of types lift to equivalences of structures is called the *structure identity principle* (SIP) in HoTT/UF ([Univalent Foundations Program, 2013](#), Section 9.8). Combining this with univalence lets us lift equalities of types to equalities of structures on these types. This was originally formalized in Agda for algebraic structures and isomorphisms by Coquand & Danielsson (2013). Recently, another variation of the SIP, due to Escardó (2019), was implemented in Cubical Agda by Angiuli *et al.* (2020). This cubical SIP extracts the pattern described here, so that a user need not repeat this construction when considering new structures. Using the cubical SIP, Angiuli *et al.* (2020) have developed a variety of more substantial examples from computer science and mathematics, including queues and finite multisets.

2.2 Inductive families

When programming with dependent types in Agda, it is very common to use inductive families as they allow the programmer to encode various information using indices in the type. The classic example is vectors—length-indexed lists—which allow the programmer to write, for example, a safe `head` function that extracts the first element of a non-empty list. While such types are ubiquitous in dependently typed programming, they also cause some headache when reasoning formally about the functions written using them. For instance, one cannot naively state associativity of concatenation for vectors because the two ways of associating concatenation lead to terms of different type. To even *state* the equation, we need to substitute along the proof of associativity for addition of natural numbers. This can quickly become complicated, making reasoning about dependently typed programs rather bureaucratic. Cubical Agda offers a solution to this as the built-in `PathP` equality is heterogeneous, making it more natural to express such equations. In this section, we will show how this helps with dependently typed programming by developing some basic results about vectors and size-indexed matrices.

Another very important example of an inductive family is the equality type. With the recently added support for inductive families to Cubical Agda, we can work with equality without losing the benefits of dependent pattern matching on the reflexivity constructor. Furthermore, we can prove that the equality type is equivalent to the `≡` type, making it possible to give computational meaning to functional extensionality and univalence expressed using the equality type.

2.2.1 Vectors

It is straightforward to define vectors the same way as in, for example, the Agda standard library. Here and in the following, we implicitly quantify over $m\ n\ k : \mathbb{N}$:

```
data Vec (A : Set ℓ) : ℕ → Set ℓ where
  []      : Vec A zero
  _::__   : (x : A) (xs : Vec A n) → Vec A (suc n)
```

We can also easily define some operations like vector concatenation:

```
_++_ : Vec A m → Vec A n → Vec A (m + n)
[]    ++ ys = ys
(x :: xs) ++ ys = x :: (xs ++ ys)
```

The fact that `_++_` is associative can be conveniently expressed using `PathP` and `+-assoc`. The proof is just a direct use of pattern matching and an inlined use of `cong`:

```

++-assoc : (xs : Vec A m) (ys : Vec A n) (zs : Vec A k) →
  PathP (λ i → Vec A (+-assoc m n k i))
    (xs ++ (ys ++ zs)) ((xs ++ ys) ++ zs)
++-assoc {m = zero} [] ys zs = refl
++-assoc {m = suc m} (x :: xs) ys zs i = x :: +-assoc xs ys zs i

```

2.2.2 Matrices

A common use of vectors is to define matrices; however, it is quite difficult to prove properties about functions defined for matrices defined this way. Another representation that is better suited for proving properties is to use functions out of a finite set of indices. To define these, we need standard finite sets which is another example of an inductive family:

```

data Fin : ℕ → Set where
  zero : Fin (suc n)
  suc : (i : Fin n) → Fin (suc n)

```

Using this, we can define two different representations of matrices:

```

FinMatrix : (A : Set ℓ) (m n : ℕ) → Set ℓ
FinMatrix A m n = Fin m → Fin n → A

VecMatrix : (A : Set ℓ) (m n : ℕ) → Set ℓ
VecMatrix A m n = Vec (Vec A n) m

```

It is straightforward to define functions going between the two representations:

```

FinVec→Vec : (Fin n → A) → Vec A n
FinVec→Vec {n = zero} xs = []
FinVec→Vec {n = suc _} xs = xs zero :: FinVec→Vec (λ x → xs (suc x))

lookup : Fin n → Vec A n → A
lookup zero (x :: xs) = x
lookup (suc i) (x :: xs) = lookup i xs

Fin→VecMatrix : FinMatrix A m n → VecMatrix A m n
Fin→VecMatrix M = FinVec→Vec (λ i → FinVec→Vec (λ j → M i j))

Vec→FinMatrix : VecMatrix A m n → FinMatrix A m n
Vec→FinMatrix M i j = lookup j (lookup i M)

```

Using `funExt`, we can prove that the functions between `FinMatrix` and `VecMatrix` cancel which gives us an equivalence of the two representations. This can then be transformed into a path by applying `ua`:

```

FinMatrix≡VecMatrix : (A : Set ℓ) (m n : ℕ) → FinMatrix A m n ≡ VecMatrix A m n
FinMatrix≡VecMatrix A m n = ua (FinMatrix≃VecMatrix A m n)

```

We can now do the same kind of transport of properties that we did for unary and binary numbers. Let us assume that we have a commutative ring $\mathbf{R}$. We write $+$ for the additive operation of $\mathbf{R}$ and the proof that it is commutative is called `commring+-comm`. It is then very easy to prove that addition of `FinMatrix` is commutative:

```
addFinMatrix : (M N : FinMatrix R m n) → FinMatrix R m n
addFinMatrix M N k l = M k l + N k l

addFinMatrixComm : (M N : FinMatrix R m n) →
  addFinMatrix M N ≡ addFinMatrix N M
addFinMatrixComm M N i k l = +-comm (M k l) (N k l) i
```

Note the inlined use of function extensionality for binary functions in `addFinMatrixComm`. Following exactly the same recipe as for transporting addition of unary numbers and the fact that it is associative to binary numbers, we can now transport addition of `FinMatrix` and the fact that it is commutative to `VecMatrix`:

```
T : Set ℓ → Set ℓ
T X = Σ[ _+_ ∈ (X → X → X) ] ((x y : X) → x + y ≡ y + x)

TVecMatrix : T (VecMatrix R m n)
TVecMatrix {m} {n} = subst T (FinMatrix≡VecMatrix R m n)
  (addFinMatrix , addFinMatrixComm)

addVecMatrix : (M N : VecMatrix R m n) → VecMatrix R m n
addVecMatrix = fst TVecMatrix

addVecMatrixComm : (M N : VecMatrix R m n) →
  addVecMatrix M N ≡ addVecMatrix N M
addVecMatrixComm = snd TVecMatrix
```

Note that there is really no added complexity here compared to the unary and binary numbers example. However, just like we get a naive addition function on binary numbers this way we get a naive one for `VecMatrix` as well. The `addVecMatrix` function transports the input matrices to `FinMatrix`, add them using `addFinMatrix`, and then transport the result back. We can of course do better and define addition for `VecMatrix` more directly by:

```
addVec : Vec R m → Vec R m → Vec R m
addVec [] [] = []
addVec (x :: xs) (y :: ys) = x + y :: addVec xs ys

addVecMatrix' : (M N : VecMatrix R m n) → VecMatrix R m n
addVecMatrix' [] [] = []
addVecMatrix' (M :: MS) (N :: NS) = addVec M N :: addVecMatrix' MS NS
```

Using the fact that `addVecMatrix` is just `addFinMatrix` with transports back and forth, we can prove that it is in fact equal to `addVecMatrix'`. The proof of this is a little bit more involved so we refer the interested reader to the formalization; however, the proof is only about 10 lines of code so it is not that complex:

`addVecMatrixPath` : $(MN : \text{VecMatrix } R \ m \ n) \rightarrow$
 $\text{addVecMatrix } MN \equiv \text{addVecMatrix}' \ MN$

Combining this with `addVecMatrixComm`, we easily get the fact that the direct definition of addition for vector matrices is commutative:

`addVecMatrixComm'` : $(MN : \text{VecMatrix } R \ m \ n) \rightarrow$
 $\text{addVecMatrix}' \ MN \equiv \text{addVecMatrix}' \ NM$

This example shows that the kind of reasoning that we did for unary and binary numbers also works for more complex data structures like matrices. Furthermore, we can—without too much effort—relate proof-oriented definitions with computation-oriented ones. In proof assistants based on regular dependent theory, this is typically not as easy, since the user has to choose a representation and then stick to it. Even further, the fact that we can use paths to reason about equalities of `FinMatrix` makes it straightforward to use function extensionality which also simplifies many proofs. If one wants to develop this in standard Agda, one instead has to resort to either using setoids or define a notion of finite functions represented by their graphs which complicates the definition considerably. For more details about various considerations when developing basic matrix operations in standard Agda, we refer the interested reader to a blog post by Wood (2019).

2.2.3 Equality

A very important inductive family is the equality type. This is also written `_≡_` in the Agda standard library, but in order to avoid confusion we call it `Eq` here:

`data Eq` { $A : \text{Set } \ell$ } ($x : A$) : $A \rightarrow \text{Set } \ell$ *where*
`reflEq` : `Eq x x`

With this definition, we can define functions by pattern matching on `reflEq` just like in regular Agda :

`ap` : $(f : A \rightarrow A') \{x \ y : A\} \rightarrow \text{Eq } x \ y \rightarrow \text{Eq } (f \ x) (f \ y)$
`ap f reflEq = reflEq`

More interestingly, we can define functions between `Eq` and paths:

`eqToPath` : $\{x \ y : A\} \rightarrow \text{Eq } x \ y \rightarrow x \equiv y$
`eqToPath reflEq = refl`
`pathToEq` : $\{x \ y : A\} \rightarrow x \equiv y \rightarrow \text{Eq } x \ y$
`pathToEq` $\{x = x\} \ p = \text{transport } (\lambda \ i \rightarrow \text{Eq } x \ (p \ i)) \ \text{reflEq}$

It is straightforward to prove that these maps cancel so that we get a path between paths and equalities:

`Path≡Eq` : $\{x \ y : A\} \rightarrow (x \equiv y) \equiv (\text{Eq } x \ y)$
`Path≡Eq = ua Path≃Eq`

This means that these types share all properties expressible in type theory. For example, we can prove function extensionality for equality by going back and forth between paths:

```

funExtEq : {B : A → Set ℓ'} {fg : (x : A) → B x} →
  ((x : A) → Eq (fx) (gx)) → Eq fg
funExtEq p = pathToEq (λ i x → eqToPath (p x) i)

```

Similarly, we can also prove univalence and define HITs using equalities instead of paths. This way, we can replace the axioms from existing HoTT Agda libraries with concrete terms. The fact that Cubical Agda now has proper support for inductive families means that these developments will be able to compute closed terms to a canonical form. We are currently experimenting with this and have written some basic examples of computing winding numbers on the circle. For details, see the `Cubical.Data.Equality` file in the `agda/cubical` repository. One should also be able to apply the techniques developed by Danielsson (2020).

2.3 Univalence for coinductive types

Coinductive types allow the direct manipulation of infinite structures without breaking the consistency of the language. However, in their treatment in `Coq` and `Agda`, reasoning about them was impeded by the inability to prove two elements equal whenever they have the same unfolding, rather than when they are the same by definition (McBride, 2009). Cubical Agda solves this by exploiting the interaction between path and projection copatterns (Abel *et al.*, 2013).

The prototypical example of a coinductive type are infinite streams, which can be declared in Agda as a `coinductive` record type with two fields: `.head` and `.tail`. A function returning a stream is then defined by explaining how it computes when applied to the projections. For example, here we define `mapS` which applies a function to every element of a stream:

```

record Stream (A : Set) : Set where
  coinductive; constructor _,_
  field
    head : A
    tail : Stream A

mapS : (A → B) → Stream A → Stream B
mapS f xs .head = f (xs .head)
mapS f xs .tail = mapS f (xs .tail)

```

The result is that `mapS f xs` by itself will not unfold further. The termination checker is happy to accept this definition as productive, since it always reaches a weak head normal form in finite time when applied to projections. As shown in the following proof of the identity law for `mapS`, Cubical Agda extends the notion of productivity by allowing the same recursion pattern also for paths between streams:

```

mapS-id : (xs : Stream A) → mapS (λ x → x) xs ≡ xs
mapS-id xs i .head = xs .head
mapS-id xs i .tail = mapS-id (xs .tail) i

```

To define a path between $(\text{mapS } (\lambda x \rightarrow x) xs)$ and xs , we introduce an interval variable i and then are left to define $(\text{mapS-id } xs i)$ of type $\text{Stream } A$, so we can proceed by copatterns and corecursion.

To convince ourselves that mapS-id defines the required path, we note that if $\text{mapS-id } xs$ is supposed to be a path between $\text{mapS } (\lambda x \rightarrow x) xs$ and xs , then the type of $(\lambda i \rightarrow \text{mapS-id } xs i . \text{head})$ should be $\text{mapS } (\lambda x \rightarrow x) xs . \text{head} \equiv xs . \text{head}$. This type in turn reduces to $xs . \text{head} \equiv xs . \text{head}$, by definition of mapS , and so the constant path suffices. A similar reasoning applies to the tail case, this time using the tail clause of mapS to realize that we need a path between $\text{mapS } (\lambda x \rightarrow x) (xs . \text{tail})$ and $xs . \text{tail}$, which we provide with a corecursive call. We give a systematic description of how we compute such *boundary* constraints from the left-hand sides of clauses in Section 4.

More generally, we can define bisimilarity as a *coinductive* record and show that two bisimilar streams are equal:

```
record  $\_ \approx \_$  (xs ys : Stream A) : Set where
  coinductive
  field
     $\approx \text{head}$  : xs . head  $\equiv$  ys . head
     $\approx \text{tail}$  : xs . tail  $\approx$  ys . tail
```

```
bisim :  $\forall \{xs\ ys : \text{Stream } A\} \rightarrow xs \approx ys \rightarrow xs \equiv ys$ 
bisim xs  $\approx$  ys i . head = xs  $\approx$  ys .  $\approx \text{head}$  i
bisim xs  $\approx$  ys i . tail = bisim (xs  $\approx$  ys .  $\approx \text{tail}$ ) i
```

Finally, we note that bisim is actually an equivalence, and so equality of streams is indeed bisimilarity:

```
path $\equiv$ bisim :  $\forall \{xs\ ys : \text{Stream } A\} \rightarrow (xs \equiv ys) \equiv (xs \approx ys)$ 
```

The *agda/cubical* library contains the complete proof, as well as a proof of the universal property of indexed M-types (Ahrens et al., 2015).

2.4 Quotient types as HITs

Another major new addition in HoTT are HITs. These are datatypes in which we can specify “higher” constructors representing nontrivial paths of the type (representing identifications of elements), in addition to the normal “point” constructors. These types enable many interesting constructions in type theory, in particular quotient types.

The addition of HITs in systems like Agda or Coq is usually done by postulating their existence; however, this suffer from the same issues, in terms of computation, as postulating the univalence axiom. Cubical Agda extends the datatype declarations of Agda to also support a general schema of HITs, so that it is not necessary to postulate their existence axiomatically.

In this section, we illustrate how we can use HITs to define quotient types in Cubical Agda. The first example of a quotient type is a very simple encoding of the integers—while this example might seem rather trivial it will help us showcase quite

a few interesting possibilities of working with HITs. The second example is a general formulation of quotient types and set quotients.

2.4.1 Integers as a HIT

The integers are often represented as $\mathbb{N} + \mathbb{N}$; however, this has the drawback that there are two zeroes (`inl 0` and `inr 0`). This is usually resolved by shifting one of them by 1 (so that for example `inl 0` represents -1 , etc.); however, this can easily lead to confusion and off-by-one errors. A better solution is to identify the two zeroes. This can be achieved with the following HIT:

```
data ℤ : Set where
  pos : (n : ℕ) → ℤ
  neg : (n : ℕ) → ℤ
  posneg : pos 0 ≡ neg 0

sucℤ : ℤ → ℤ
sucℤ (pos n)      = pos (suc n)
sucℤ (neg zero)   = pos 1
sucℤ (neg (suc n)) = neg n
sucℤ (posneg i)   = pos 1
```

This type is similar to $\mathbb{N} + \mathbb{N}$, except that there is also a *path* constructor `posneg` which identifies the two zeroes. For an element i of the interval type, `posneg i` is an integer which reduces to `pos 0` in case $i = i0$ and `neg 0` in case $i = i1$. These so-called *boundary conditions* of `posneg` have to be respected by any function on $\mathbb{Z}$. For example, the successor function on $\mathbb{Z}$ can be written as above. The final case maps the path constructor constantly to `pos 1` which is accepted by Cubical Agda as the following equations hold definitionally:

$$\text{suc}\mathbb{Z} (\text{pos } 0) = \text{suc}\mathbb{Z} (\text{neg } 0) = \text{pos } 1.$$

It is direct to define an inverse to `sucℤ` (i.e., the predecessor function) and hence get an equivalence from $\mathbb{Z}$ to $\mathbb{Z}$ which, combined with `ua`, gives a nontrivial path from $\mathbb{Z}$ to $\mathbb{Z}$. Transporting along this path applies the successor function:

```
sucPathℤ : ℤ ≡ ℤ
sucPathℤ = isoToPath (iso sucℤ predℤ sucPredℤ predSucℤ)
```

We can also define addition and prove that addition with a fixed number is an equivalence; however, this takes a bit of work as we need to define subtraction and prove that it is the inverse of addition. Using univalence, we can take a shortcut and define an alternative addition function so that addition with a fixed number is automatically an equivalence. Consider the following path equality that composes `sucPathℤ` with itself n times:

```
addEq : ℕ → ℤ ≡ ℤ
addEq zero   = refl
addEq (suc n) = addEq n • sucPathℤ
```

Similarly, we can define a path composing the predecessor path with itself n times. By transporting along these paths, we get an addition function:

```

addℤ : ℤ → ℤ → ℤ
addℤ m (pos n)    = transport (addEq n) m
addℤ m (neg n)    = transport (subEq n) m
addℤ m (posneg _) = m

```

Using that transporting along a path is an equivalence, we get that addition by a fixed number is an equivalence.

```

isEquivAddℤ : (m : ℤ) → isEquiv (λ n → addℤ n m)
isEquivAddℤ (pos n)    = isEquivTransport (addEq n)
isEquivAddℤ (neg n)    = isEquivTransport (subEq n)
isEquivAddℤ (posneg i) = isEquivTransport refl

```

2.4.2 General quotient types and set quotients

In Agda and other dependently typed programming languages, quotient type could so far only be defined axiomatically. Here, we show how to define them as a HIT in Cubical Agda.

The first attempt is the following definition:

```

data _/_ (A : Set ℓ) (R : A → A → Set ℓ) : Set ℓ where
  [ ] : A → A / R
  eq  : (a b : A) → R a b → [ a ] ≡ [ b ]

```

This type has a constructor for mapping elements of A to the quotient by R and an equality identifying the image of each pair of related elements. However, this is not exactly what we want because the resulting quotient type might have a too complex notion of equality. For example, if we use this construction to quotient `Unit` by the total relation, then we will get a type with a point `[ tt ]` and an identification of this point with itself. We would expect this to be equivalent to `Unit`; however, it is in fact equivalent to the HIT circle that we will discuss in [Section 2.5.1](#). As `Unit` and the circle do not satisfy the same properties, they are not equivalent; the loop space of `Unit`, here the type of paths from the only element to itself, is contractible, while the loop space of the circle is $\mathbb{Z}$ ([Univalent Foundations Program, 2013](#)).

We get the expected notion of quotients if we switch to *set* quotients. We add another higher constructor that eliminates all of the higher-dimensional structure from the quotient type, in other words, we *set truncate* the type:

```

data _/_ (A : Set ℓ) (R : A → A → Set ℓ) : Set ℓ where
  [ ] : A → A / R
  eq  : (a b : A) → (r : R a b) → [ a ] ≡ [ b ]
  trunc : (x y : A / R) → (p q : x ≡ y) → p ≡ q

```

This makes the quotient into a *recursive* HIT as the `trunc` constructor quantifies over elements of the type that we are constructing. It forces the quotient to be a set in the sense of satisfying the *uniqueness of identity proofs* (UIP) principle, in other words, any two proofs of equality of members of A / R are equal. Thanks to `trunc`, we can prove the universal property of set quotients:

`setQuotUniversal` : $\{A\ B : \mathbf{Set}\ \ell\} \{R : A \rightarrow A \rightarrow \mathbf{Set}\ \ell\} \rightarrow \mathbf{isSet}\ B \rightarrow$
 $(A / R \rightarrow B) \simeq (\Sigma[f \in (A \rightarrow B)] (\forall a\ b \rightarrow R\ a\ b \rightarrow f\ a \equiv f\ b))$

This says that maps out of the set quotient is the same as maps sending related elements to equal elements in the quotient (assuming that the image satisfies UIP). If we furthermore assume that R is a propositional equivalence relation, then the set quotients are effective, in the sense that if $[a] \equiv [b]$ then also $R\ a\ b$. As an interesting application, we could for example define the positive fractions as a quotient of $\mathbb{N} \times \mathbb{N}$ by relating (n_1, d_1) and (n_2, d_2) if $(n_1 \cdot (1 + d_2) \equiv n_2 \cdot (1 + d_1))$.

2.5 Synthetic homotopy theory in Cubical Agda

One of the main applications of HITs in HoTT is the ability to reason *synthetically* about topological spaces inside type theory. This means that we can define topological spaces (like spheres, tori, etc.) as datatypes and reason about them using functional programming. The semantic justification for this is the standard model in Kan simplicial sets, a combinatorial representation of topological spaces (Kapulkin & Lumsdaine, 2012; Lumsdaine & Shulman, 2017). We will discuss how cubical type theory relates to Kan simplicial sets in Section 6. In this section, we illustrate how we can do synthetic homotopy theory in Cubical Agda by proving that the torus is equivalent to two circles.

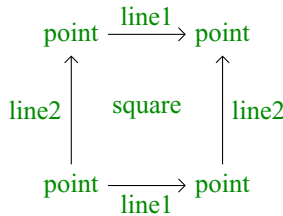
2.5.1 The torus and two circles

We can define circle and torus as the following HITs:

```
data S1 : Set where
  base : S1
  loop  : base ≡ base

data Torus : Set where
  point  : Torus
  line1  : point ≡ point
  line2  : point ≡ point
  square : PathP (λ i → line1 i ≡ line2 i) line2 line2
```

The idea is that the circle, S^1 , is generated by a `base` point and a nontrivial path constructor `loop` connecting `base` to itself. The `Torus` on the other hand also has a base `point` with two nontrivial path constructors connecting it to itself and a `square` relating the two paths. This square can be illustrated by:



The idea is that the `square` constructor identifies `line2` with itself over an identification of `line1` with itself. This has the effect of identifying the opposite sides of the square, making it into a torus (imagine the square being a sheet of soft paper that one folds so that the opposite sides match).

As we demonstrate in the following, a torus is equivalent to the product of two circles:

<code>t2c : Torus → S¹ × S¹</code>	<code>c2t : S¹ × S¹ → Torus</code>
<code>t2c point = (base , base)</code>	<code>c2t (base , base) = point</code>
<code>t2c (line1 i) = (loop i , base)</code>	<code>c2t (loop i , base) = line1 i</code>
<code>t2c (line2 j) = (base , loop j)</code>	<code>c2t (base , loop j) = line2 j</code>
<code>t2c (square i j) = (loop i , loop j)</code>	<code>c2t (loop i , loop j) = square i j</code>

The functions back and forth are directly definable by pattern matching. As a consequence, proving that they are mutually inverse is trivial, and we get an equality between the two types:

```

c2t-t2c : (t : Torus) → c2t (t2c t) ≡ t
c2t-t2c point = refl
c2t-t2c (line1 _) = refl
c2t-t2c (line2 _) = refl
c2t-t2c (square _ _) = refl

```

```

t2c-c2t : (p : S1 × S1) → t2c (c2t p) ≡ p
t2c-c2t (base , base) = refl
t2c-c2t (base , loop _) = refl
t2c-c2t (loop _ , base) = refl
t2c-c2t (loop _ , loop _) = refl

```

```

Torus ≡ S1 × S1 : Torus ≡ S1 × S1
Torus ≡ S1 × S1 = isoToPath (iso t2c c2t t2c-c2t c2t-t2c)

```

This is a rather elementary result in topology. However, it had a surprisingly nontrivial proof in HoTT because of the lack of definitional computation for higher constructors (Licata & Brunerie, 2015; Sojakova, 2016). With the additional definitional computation rules of Cubical Agda, this proof is now almost entirely trivial.

2.5.2 Further synthetic homotopy theory in Cubical Agda

The `agda/cubical` library contains several further results from synthetic homotopy theory. For instance, we have a direct proof that the fundamental group of the circle is $\mathbb{Z}$, inspired by Licata & Shulman (2013). Combined with the above characterization of the torus, it proves that the fundamental group of the `Torus` is $\mathbb{Z} \times \mathbb{Z}$. The fact that univalence and HITs compute in Cubical Agda lets us then compute winding numbers of iterated loops around the circle and torus.

The library also features more substantial results: a proof that $\mathbb{S}^3$, that is, the four dimensional sphere, is equivalent to the join of two circles, and a proof that the total space of the Hopf fibration is $\mathbb{S}^3$ (Mörtberg & Pujet, 2020). We also have a definition of the “Brunerie

number”: a number $n \in \mathbb{Z}$ such that $\pi_4(\mathbb{S}^3) \simeq \mathbb{Z}/n\mathbb{Z}$.³ However, despite considerable efforts we have not been able to reduce n to a normal form yet; even though the absolute value of the expected result is just 2 as proved by Brunerie (2016).

3 Making Agda cubical

In the remainder of the paper, we will describe how Agda was extended to become cubical. The key additions to Agda are

1. The interval and path types (Section 3.1).
2. Generalized transport, `transp` (Section 3.2).
3. Partial elements (Section 3.3).
4. Homogeneous composition, `hcomp` (Section 3.4).
5. Higher inductive types (Section 4).
6. `Glue` types (Section 5).

We have already discussed the first two points in some detail in the examples; however, the implementation of the `transp` operation is especially interesting as it is what makes Cubical Agda compute. This operation is defined by cases on the type formers of Agda ; a contribution of this paper is the extension of these to types that are present in Agda but not covered by Cohen *et al.* (2018), namely, record and coinductive types. Furthermore, the way the `hcomp` operation works in Cubical Agda differs from Coquand *et al.* (2018) in a subtle way which enables us to optimize the `transp` operation for `Glue` types. These types are what allows us to give computational content to univalence. By optimizing how their composition operation computes, we obtain simpler and more efficient proofs of univalence. As discussed in Section 6, this also has meta-theoretical consequences for the canonicity theorem for HITs.

3.1 The interval and path types

The first thing Cubical Agda adds is an interval type `I`. Then, we add the `PathP` types that behave like function types out of the interval, but with fixed endpoints. Note that the interval in Cubical Agda is not inductively defined, so we cannot pattern match on it. This follows from the intuition that path types are *continuous* functions from the interval into a space, so that they cannot provide arbitrarily different results for `i0` and `i1`.

3.2 Generalized transport

The next key thing that Cubical Agda adds is the generalized transport operation:

$$\text{transp} : (A : I \rightarrow \text{Set } \ell) \rightarrow I \rightarrow A \text{ i0} \rightarrow A \text{ i1}$$

Given a type line $A : I \rightarrow \text{Set } \ell$ and an element at end `A i0`, the `transp` operation gives an element at `A i1`, the other end of the line. This is generalized compared to regular transport

³ For details see <https://github.com/agda/cubical/blob/master/Cubical/Experiments/Brunerie.agda>.

in the sense that `transp` lets us specify where it should behave as the identity function. In particular, there is an additional side condition to be satisfied for `transp A r a` to type-check, which is that A should be a constant function whenever the constraint $r = \mathbf{i1}$ is satisfied. When r is equal to `i1`, the `transp` function will compute as the identity function:

$$\text{transp } A \mathbf{i1} \ a = a$$

and this would not be sound in general if A was allowed to be a more complex line that is nonconstant when $r = \mathbf{i1}$. In case $r = \mathbf{i0}$ there is nothing to check, thus, `transp A i0 a` is well formed for any A , as in the definition of `transport A a`.

Internally, the `transp` operation computes differently for each of the type formers of Agda . We will show how this works in the special case of `transport`, but the general `transp` operation is not much more complicated. For a detailed type-theoretic presentation of these definitions, see Huber (2017). This formulation of the computation rules for cubical type theory is based on a variation of the `comp` operation of Cohen *et al.* (2018) that was introduced in Coquand *et al.* (2018) in order to support HITs.

3.2.1 Function types

Given two type lines $A B : \mathbf{l} \rightarrow \mathbf{Set}$, we seek to transport a function $f : A \mathbf{i0} \rightarrow B \mathbf{i0}$ to a function $A \mathbf{i1} \rightarrow B \mathbf{i1}$. To this end, we compose backward transport $A \mathbf{i1} \rightarrow A \mathbf{i0}$ along A , function f , and forward transport $B \mathbf{i0} \rightarrow B \mathbf{i1}$ along B :

$$\begin{aligned} \text{transportFun} &: (A B : \mathbf{l} \rightarrow \mathbf{Set}) \rightarrow (A \mathbf{i0} \rightarrow B \mathbf{i0}) \rightarrow (A \mathbf{i1} \rightarrow B \mathbf{i1}) \\ \text{transportFun } A B f &= \text{transport } (\lambda i \rightarrow B i) \circ f \circ \text{transport } (\lambda i \rightarrow A (\sim i)) \end{aligned}$$

By evaluating `transport` $(\lambda i \rightarrow A i \rightarrow B i) f$, we can see that the definition of `transportFun A B f` is definitionally the same as the internal definition for how `transp` computes in Cubical Agda :

$$\begin{aligned} \text{transportFunEq} &: (A B : \mathbf{l} \rightarrow \mathbf{Set}) \rightarrow (f : A \mathbf{i0} \rightarrow B \mathbf{i0}) \rightarrow \\ &\quad \text{transportFun } A B f \equiv \text{transport } (\lambda i \rightarrow A i \rightarrow B i) f \\ \text{transportFunEq } A B f &= \text{refl} \end{aligned}$$

The definition for dependent functions is very similar, except that some extra work is required to correct the type in the outer `transport`. This definition clarifies why we need to consider `transp` and not just the simpler `transport` operation:

$$\begin{aligned} \text{transportPi} &: (A : \mathbf{l} \rightarrow \mathbf{Set}) (B : (i : \mathbf{l}) \rightarrow A i \rightarrow \mathbf{Set}) \\ &\quad \rightarrow ((x : A \mathbf{i0}) \rightarrow B \mathbf{i0} x) \\ &\quad \rightarrow ((x : A \mathbf{i1}) \rightarrow B \mathbf{i1} x) \\ \text{transportPi } A B f &= \lambda (x : A \mathbf{i1}) \rightarrow \\ &\quad \text{transport } (\lambda j \rightarrow B j (\text{transp } (\lambda i \rightarrow A (j \vee \sim i)) j x)) \\ &\quad (f(\text{transport } (\lambda i \rightarrow A (\sim i)) x)) \end{aligned}$$

If we would have used the same definition as for nondependent functions, the outer `transport` would have been ill-typed. The reason is that f has a dependent type, meaning that $f x'$ has type $B x'$ for $x' := \text{transport } (\lambda i \rightarrow A (\sim i)) x$. The first argument of the outer `transport` must hence be a line between $B \mathbf{i0} x'$ and $B \mathbf{i1} x$. This line is constructed by abstracting over j and considering $B j (\text{transp } (\lambda i \rightarrow A (j \vee \sim i)) j x)$. When j is `i0` this is

indeed $B \text{ i0 } x'$ and when j is i1 this is $B \text{ i1 } x$ by virtue of `transp` being the identity function when applied to i1 .

3.2.2 Records and coinductive types

For record types, the `transport` operation is computed pointwise, that is, independently for every field. The only subtlety is when the record is dependent, in which case a similar type correction has to be done as for dependent functions.

As coinductive types in Agda are just record types, these are handled in the same way. In this case, however, we have to consider the issue of productivity, which is taken care of by how `transport` for record types unfolds only when projected from. Analogously to the `Stream` example from Section 2.3, we have that `transport` $(\lambda i \rightarrow \text{Stream } A) xs$ will not reduce further, while if we apply `.tail` to it we get `transport` $(\lambda i \rightarrow \text{Stream } A) (xs \text{ .tail})$.⁴ Such controlled unfolding generally leads to smaller normal forms, so Cubical Agda adopts it for record types in general. The same kind of controlled unfolding is also implemented for other “negative” types like function and path types.

3.2.3 Datatypes

The `transport` operation for inductive datatypes without parameters, for instance, the natural numbers, is trivial as they cannot vary along the interval:

$$\text{transport } (\lambda i \rightarrow \mathbb{N}) x = x$$

This would not work for inductive types with parameters like the disjoint union $A + B$ for which the `transport` operation would need to reduce to the `transport` operation in A or B depending on the argument:

```
transportSum : (A B : I → Set) → A i0 + B i0 → A i1 + B i1
transportSum A B (inl x) = inl (transport (λ i → A i) x)
transportSum A B (inr x) = inr (transport (λ i → B i) x)
```

The `transp` operation for HITs is a bit more involved. It also computes by cases on the argument, but for the higher constructors some extra care has to be taken. In particular, complicated parameterized HITs, like pushouts, require additional endpoint corrections (Coquand *et al.*, 2018, Section 3.3.5).

3.2.4 Inductive families

In the case of inductive families, we have an extra complication: constructors only target specific indexes. For example, it is not clear how to proceed when reducing `transport` $(\lambda i \rightarrow \text{Vec } A (p \text{ i})) []$, as `[]` might not fit the expected result type of `Vec A (p i1)`. Moreover, we not only have to care about the endpoint, but we should also keep track of the path p , as it might contain computationally relevant information.

To address this problem, we adapt the strategy of Cavallo & Harper (2019), adding a constructor to each inductive family to represent a residual index transport. In the case

⁴ Productivity for the case of dependent records then relies on the type of later fields only being able to depend on earlier fields.

of vectors, we have a constructor⁵ $\text{transpX}_{\text{Vec}} p r u_0 : \text{Vec } A (p \text{ i1})$ for $p : \mathbb{I} \rightarrow \mathbb{N}$, $r : \mathbb{I}$ and $u_0 : \text{Vec } A (p \text{ i0})$, such that p is constant when $r = \text{i1}$. Like for transp , we have that $\text{transpX}_{\text{Vec}} p \text{ i1 } u_0$ reduces to u_0 . We can then transport constructors like so:

$$\begin{aligned} \text{transport } (\lambda i \rightarrow \text{Vec } (A \text{ i}) (p \text{ i})) [] &= \text{transpX}_{\text{Vec}} (\lambda i \rightarrow p \text{ i}) \text{ i0 } [] \\ \text{transport } (\lambda i \rightarrow \text{Vec } (A \text{ i}) (p \text{ i})) (x :: xs) &= \text{transpX}_{\text{Vec}} (\lambda i \rightarrow p \text{ i}) \text{ i0} \\ &\quad (\text{transport } (\lambda i \rightarrow A \text{ i}) x :: \text{transport } (\lambda i \rightarrow \text{Vec } (A \text{ i}) m) xs) \end{aligned}$$

In the clause for $[]$, the typing lets us assume that $p \text{ i0}$ is equal to 0 , while in the second clause the typing implies $p \text{ i0}$ is equal to $\text{succ } m$ where m is the length of $xs : \text{Vec } (A \text{ i0}) m$. In both cases, $\text{transpX}_{\text{Vec}}$ lets us produce a result at the desired index by storing the path p . The full reduction algorithm for transp on inductive families is an adaptation of the one for the coercion operator from Cavallo & Harper (2019).

In Section 4, we will show how definitions by pattern matching can be extended to cover the extra transpX constructor, without the user needing to specify a clause for it.

3.2.5 Path types

For path types, we will need a new operation to provide the computation rules for transport as we need some way to record the endpoints of the path after transporting it. Indeed, consider the following naïve definition:

$$\begin{aligned} \text{transportPath} : (A : \mathbb{I} \rightarrow \text{Set}) (x y : (i : \mathbb{I}) \rightarrow A \text{ i}) \rightarrow x \text{ i0} \equiv y \text{ i0} \rightarrow x \text{ i1} \equiv y \text{ i1} \\ \text{transportPath } A x y p = \lambda i \rightarrow \text{transport } (\lambda j \rightarrow A \text{ j}) (p \text{ i}) \end{aligned}$$

This might look plausible as a definition, but the resulting path does not have the correct boundary. When i is i0 , for instance, the left boundary is $\text{transport } (\lambda j \rightarrow A \text{ j}) (x \text{ i0})$ and not just $x \text{ i1}$. Note that these elements are equal up to a path (using $\lambda k \rightarrow \text{transp } (\lambda j \rightarrow A (j \vee k)) k (x \text{ k})$), so what we need is a way to compose the result with this path in order to correct the endpoints. To do this, we introduce the homogeneous composition operation (hcomp) that generalizes binary composition of paths to n -ary composition of higher-dimensional cubes.

3.3 Partial elements

In order to describe the homogeneous composition operation, we need to be able to write partially specified n -dimensional cubes, that is, cubes where some faces are missing. Given an element of the interval $r : \mathbb{I}$, there is a new primitive predicate $\text{lsOne } r$ which represents the constraint $r = \text{i1}$. This comes with a proof 1=1 that i1 is in fact equal to i1 , that is, $\text{1=1} : \text{lsOne } \text{i1}$. The type $\text{lsOne } (i \vee \sim i)$ corresponds to the formula $(i = \text{i1}) \vee (i = \text{i0})$ which represents the two endpoints of the line specified by i , so by considering formulas made out of more variables we can specify the boundary of cubes. The type $\text{lsOne } r$ is also proof-irrelevant, meaning that any two of its elements are definitionally equal.

Building on lsOne , we have extended Cubical Agda with partial cubical types, written $\text{Partial } r A$. The idea is that $\text{Partial } r A$ is the type of cubes in A that are only defined when

⁵ The X in transpX stands for “index.”

$\text{IsOne } r$ holds.⁶ Concretely, $\text{Partial } r A$ is a special version of the function space $\text{IsOne } r \rightarrow A$ with a more extensional equality: two of its elements are considered judgmentally equal if they represent the same subcube of A . Concretely, they are equal whenever they reduce to equal terms for all the possible assignment of variables that make r equal to i1 . An example of where this much extensionality is useful is the definition of hfill in Section 3.4.

Elements of these partial cubical types are introduced using pattern-matching lambdas. For this purpose, Cubical Agda supports a new form of patterns, here $(i = \text{i0})$ and $(i = \text{i1})$, that specify the cases where $\text{IsOne } (i \vee \sim i)$ is true. Similarly to pattern matching on an inductive family, some variables from the context might get refined, in this case i , even if otherwise we would not be able to pattern match on them:

```
partialBool : ∀ i → Partial (i ∨ ∼ i) Bool
partialBool i = λ { (i = i0) → true ; (i = i1) → false }
```

The term `partialBool` should be thought of as a boolean with different values when i is i0 and when i is i1 . This is hence just the endpoints of a line and there is no way to connect them, since `true` is not equal to `false`. The pattern-matching cases must match the interval expression in the type (under the image of IsOne) and if there are overlapping cases then they must agree up to definitional equality. Furthermore, $\text{IsOne } \text{i0}$ is actually absurd and lets us define an empty partial element, also known as an “empty system” (Cohen *et al.*, 2018, Section 4.2):

```
empty : Partial i0 A
empty = λ { () }
```

Cubical Agda also has cubical subtypes as in Cohen *et al.* (2018); given $A : \text{Set } \ell$ and $r : \mathbb{I}$ and $u : \text{Partial } r A$, we can form the type $A [r \mapsto u]$. A term v of this type is a term of type A that is definitionally equal to u when $\text{IsOne } r$ is satisfied.⁷ Any term $u : A$ can be seen as a term of type $A [r \mapsto u]$ that agrees with itself when $\text{IsOne } r$. This observation is incarnated in the introduction principle `inS`:

```
inS : {r : I} (a : A) → A [r ↦ (λ _ → a)]
```

We can also forget that a partial element agrees with u when $\text{IsOne } r$ holds. This insight is manifest in the subsumption principle `outS`:

```
outS : {r : I} {u : Partial r A} → A [r ↦ u] → A
```

We have that both $\text{outS } (\text{inS } v) = v$ and $\text{inS } (\text{outS } v) = v$ hold if well typed. Moreover, $\text{outS } \{r\} \{u\} v$ will reduce to $u \text{ i1}$ when $r = \text{i1}$.

With all of this cubical infrastructure, we can now describe the `hcomp` operation.

3.4 Homogeneous composition

The homogeneous composition operation generalizes binary composition of paths so that we can compose multiple composable cubes:

⁶ `Partial` is somewhat analogous to constrained set $P \Rightarrow A = \{a \in A \mid P\}$ where $P = \text{IsOne } r$, only that the proof of P matters.

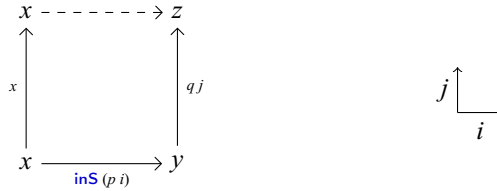
⁷ In the set-theoretic analogy, $A [r \mapsto u] = \{a \in A \mid \text{if } r \text{ then } (a = u)\} \subseteq A$, given $u \in \{a \in A \mid r\}$. We have $a \in A [r \mapsto \lambda _ \rightarrow a]$ always for $a \in A$.

$\text{hcomp} : \{r : \mathbb{I}\} (u : \mathbb{I} \rightarrow \text{Partial } r A) (u_0 : A [r \mapsto u \text{ i0}]) \rightarrow A$

When calling $\text{hcomp } u u_0$, Cubical Agda makes sure that u_0 agrees with $u \text{ i0}$ on r ; this is specified in the type of u_0 . The idea is that u_0 is the base and u specifies the sides of an open box where the side opposite of u_0 is missing. The hcomp operation then gives us the missing side opposite of u_0 , which we refer to as the *lid* of the open box. For example, binary composition of paths can be written as:

$\bullet_ : \{x y z : A\} \rightarrow x \equiv y \rightarrow y \equiv z \rightarrow x \equiv z$
 $\bullet_ : \{x = x\} p q i = \text{hcomp } (\lambda j \rightarrow \lambda \{ (i = \text{i0}) \rightarrow x ; (i = \text{i1}) \rightarrow q j \}) (\text{inS } (p i))$

Pictorially, we are given $p : x \equiv y$ and $q : y \equiv z$, and the composite of the two paths is obtained by computing the dashed lid at the top of the following square:



As we are constructing a path from x to z along i , we have $i : \mathbb{I}$ in context and put $\text{inS } (p i)$ as bottom line. The direction j is abstracted in the first argument to hcomp and we use pattern matching to specify the sides.

We can also define homogeneous filling of open boxes as:

$\text{hfill} : \{r : \mathbb{I}\} (u : \mathbb{I} \rightarrow \text{Partial } r A) (u_0 : A [r \mapsto u \text{ i0}]) \rightarrow \mathbb{I} \rightarrow A$
 $\text{hfill } \{r = r\} u u_0 i =$
 $\text{hcomp } (\lambda j \rightarrow \lambda \{ (r = \text{i1}) \rightarrow u (i \wedge j) \text{ 1=1} ; (i = \text{i0}) \rightarrow \text{outS } u_0 \})$
 $(\text{inS } (\text{outS } u_0))$

When i is i0 this is just $\text{outS } u_0$ and when i is i1 it is $\text{hcomp } (\lambda j \rightarrow \lambda \{ (r = \text{i1}) \rightarrow u j \text{ 1=1} \}) u_0$ because the absurd face $(\text{i0} = \text{i1})$ gets filtered out. By the extensionality of partial elements, this gives a line along i between $\text{outS } u_0$ and $\text{hcomp } u u_0$ which geometrically corresponds to the filling of an open box as it connects the base with the lid computed using hcomp . The elimination followed by introduction in $\text{inS } (\text{outS } u_0)$ might look redundant, but it is necessary because the sides of this composition are defined on $r \vee \sim i = \text{i1}$, while u_0 belongs to a subtype specified on r . In the special case when q is refl , the filler of the above square gives us a direct cubical proof that composing p with refl is p :

$\text{compPathRefl} : \{x y : A\} (p : x \equiv y) \rightarrow p \bullet \text{refl} \equiv p$
 $\text{compPathRefl } \{x = x\} \{y = y\} p j i =$
 $\text{hfill } (\lambda _ \rightarrow \lambda \{ (i = \text{i0}) \rightarrow x ; (i = \text{i1}) \rightarrow y \}) (\text{inS } (p i)) (\sim j)$

This way, we can do even more equality reasoning by directly working with higher-dimensional cubes.

By combining hcomp and transp , we can define the heterogeneous composition operation of Cohen *et al.* (2018):

$\text{comp} : (A : \mathbb{I} \rightarrow \text{Set } \ell) \{r : \mathbb{I}\} (u : (i : \mathbb{I}) \rightarrow \text{Partial } r (A i))$
 $(u0 : A \text{ i0} [r \mapsto u \text{ i0}]) \rightarrow A \text{ i1}$

```

comp A {r = r} u u0 =
  hcomp (λ i → λ { (r = i1) → transp (λ j → A (i ∨ j)) i (u _ 1=1) })
    (inS (transport (λ i → A i) (outS u0)))

```

With the `comp` operation, we can then finally give the definition of `transp` for path types:

```

transportPath : (A : I → Set) (x y : (i : I) → A i) → x i0 ≡ y i0 → x i1 ≡ y i1
transportPath A x y p =
  λ i → comp A (λ j → λ { (i = i0) → x j ; (i = i1) → y j }) (inS (p i))

```

The computation rules for `hcomp` are also defined by cases on the type formers of Agda, just like for `transp`. These are all quite direct to define and we refer the interested reader to Huber (2017) for details. We will note, however, that for HITs and inductive families `hcomp (λ i → λ { (r = i1) → u }) u0` only reduces to `u[i1/i]` when $r = i1$ and is to be considered a canonical element otherwise. Therefore, functions defined by pattern matching on a HIT also have to make progress when provided an element built with `hcomp`. We will often refer to such an element as `hcomp r u u0` for ease of notation. The next section describes how this can be achieved, in the context of a core type theory for Cubical Agda.

4 Pattern matching with HITs and inductive families

Our main technical contribution is an elaboration algorithm for (co)pattern-matching definitions in the presence of HITs and path applications. Following Cockx & Abel (2018), we formulate our algorithm as a translation from (co)pattern-matching clauses to case trees. The main challenges are generating the computational behavior on `hcomp` elements of HITs, `transpX` elements of inductive families, and making sure clauses for path constructors agree with what the function does at the endpoints of the path. In this section, we will consider the equality type as our only inductive family to simplify the presentation of the algorithm, while still being able to illustrate the relevant issues.

4.1 Elaboration by example

Here, we illustrate by example how we can handle the cases for `hcomp` and `transpX`.

Consider the function `c2t` from Section 2.5.1, it is defined by four clauses, which pattern-match on a pair of elements of the circle. Recalling that `hcomp r u u0` is also a canonical element of the circle, we can see that we additionally have to cover the following cases:

```

c2t (hcomp r u u0 , y)           = ?0
c2t (base          , hcomp r u u0) = ?1
c2t (loop i        , hcomp r u u0) = ?2

```

We can cover the first by setting:

```

?0 := hcomp (λ j (r = i1) → c2t (u j 1=1, y)) (c2t (u0, y))

```

which not only produces an element of the right type but also satisfies $?0 = c2t (u i 1=1, y)$ when $r = i1$, which is required to preserve the equality `hcomp i1 u u0 = u i 1=1`. The case

?1 can be solved analogously, while ?2, since it matches on `loop`, has additional constraints: ?2 should be equal to `c2t (base, hcomp r u u0)` whenever $i = i0$ or $i = i1$. We can satisfy all of these constraints at once by including them in the composition, that is, setting:

$$?2 := \text{hcomp } (\lambda j \rightarrow \lambda \left\{ \begin{array}{l} (r = i1) \rightarrow \text{c2t } (\text{loop } i, u j \text{ } 1=1) \\ (i = i0) \rightarrow \text{c2t } (\text{base}, \text{hcomp } r u u_0) \\ (i = i1) \rightarrow \text{c2t } (\text{base}, \text{hcomp } r u u_0) \end{array} \right\}) (\text{c2t } (\text{loop } i, u_0))$$

where all the components of the partial element match up because they are all different specializations of `c2t (loop i, hcomp r u u0)` under the different boundary conditions. In the general case, the return type of the function can depend on the HIT argument, so a heterogeneous composition will be necessary.

To illustrate how to handle `transpX` elements, let us look at an artificially constrained proof of symmetry:

```
sym0 : (x : ℕ) → Eq x 0 → Eq 0 x
sym0 .0 reflEq = reflEq
```

Ideally, we would only have to handle the following extra clause involving `transpX`:

$$\text{sym0 } x (\text{transpX } p r t) = ?0$$

However, we find ourselves stuck because t has type `Eq x (p i0)` so we cannot recurse with `sym0 x t`, as that is not well typed. To get around this, we can match with `reflEq` against t , which gives us this clause:

$$\text{sym0 } x (\text{transpX } p r \text{ reflEq}) = ?0$$

Now we know from the typing that p is a path connecting x to `0`, so we can use it to solve our goal with a transport:

$$?0 := \text{transp } (\lambda i \rightarrow \text{Eq } 0 (p (\sim i))) r (\text{sym0 } 0 \text{ reflEq})$$

It is not by chance that we were able to rely on the value of `sym0` at `reflEq` to solve this goal, but rather an instance of the specialization by unification technique which is used to translate, under certain conditions, definitions by clauses into functions defined solely by eliminators (Cockx & Devriese, 2018). We refer to the algorithm in Section 4.3.3 for the general case, including how to handle remaining patterns `transpX p r (hcomp s w w0)` and `transpX p r (transpX q s t1)`.

4.2 Syntax of the core type theory

We recall some definitions from Cockx & Abel (2018) extended to allow for the new cubical primitives.

Expressions (Figure 1) are given in spine-normal form, so that the head symbol of an application is easily accessible. Rather than adding the cubical types and operations described in Section 3 as new expression formers, we subsume them under `f ē` and `c ēc`. This happens also in the implementation of Agda, thanks to the preexisting support for built-ins and primitives.

We include a universe level ω for types like `ℕ` which do not support `transp` or `hcomp`. Eliminations e include, beyond function application to u and projections `.π`, *path*

x, i		variables
ℓ	$::= n \mid \omega$	universe levels
A, B, u, v	$::= w$	weak head normal form
	$\mid \mathbf{f} \bar{e}$	defined function or primitive applied to eliminations
	$\mid x \bar{e}$	variable applied to eliminations
	$\mid \mathbf{c} \bar{e}_{\mathbf{c}}$	constructor applied to eliminations
W, w	$::= (x : A) \rightarrow B$	dependent function type
	$\mid \mathbf{Set}_{\ell}$	universe ℓ
	$\mid \mathbf{D} \bar{u}$	datatype fully applied to parameters
	$\mid \mathbf{R} \bar{u}$	record type fully applied to parameters
	$\mid \lambda x. u$	lambda abstraction
e	$::= e_{\mathbf{c}}$	elimination for constructors
	$\mid \mathbf{\pi}$	projection
$e_{\mathbf{c}}$	$::= u$	application
	$\mid @_{u_0, u_1} v$	path application

Fig. 1. Syntax of terms.

applications $@_{u_0, u_1} v$. Path applications to interval element v are annotated by the endpoints u_0 and u_1 used for reduction, in case v becomes $\mathbf{i0}$ or $\mathbf{i1}$.

In contrast to ordinary applications, path applications of stuck terms can reduce; for instance, $x @_{u_0, u_1} \mathbf{i0}$ reduces to u_0 . Thus, variable eliminations $x \bar{e}$ are not necessarily in weak head normal form. Thanks to HITs, this is not even the case for constructor applications; $\mathbf{c} \bar{e}$ might also reduce!

Binary application $\boxed{u e}$ is defined as a partial function on the syntax, by β reduction $(\lambda x. u) v = u[v/x]$ in case of abstractions, or by accumulating eliminations $(x \bar{e}) e = x(\bar{e}, e)$, $(\mathbf{f} \bar{e}) e = \mathbf{f}(\bar{e}, e)$, $(\mathbf{c} \bar{e}) e_{\mathbf{c}} = \mathbf{c}(\bar{e}, e_{\mathbf{c}})$, and otherwise it is undefined.

Patterns are augmented with path application *copatterns*, also in the spine for constructors:

p	$::= x$	variable pattern
	$\mid \mathbf{c} \bar{q}_{\mathbf{c}}$	fully applied constructor pattern
	$\mid \lfloor \mathbf{c} \rfloor \bar{q}_{\mathbf{c}}$	forced constructor pattern
	$\mid \lfloor u \rfloor$	forced argument
	$\mid \emptyset$	absurd pattern
q	$::= q_{\mathbf{c}}$	copattern for constructors
	$\mid \mathbf{\pi}$	projection copattern
$q_{\mathbf{c}}$	$::= p$	application copattern
	$\mid @_{u_0, u_1} i$	path application copattern

Note that we retain the boundary annotations of path applications even in patterns, since we convert copatterns to eliminations, denoted as $\lceil q \rceil$, during type and coverage checking for case trees.

We write $\boxed{\text{PV}(\bar{q})}$ for the set of variables appearing as variable patterns x or as i in a path application copattern. We will also often drop the subscript from $e_{\mathbf{c}}$ and $q_{\mathbf{c}}$.

s	$::= \ominus$	status: unchecked
	$ \oplus$	status: checked
$decl^s$	$::= \text{data } D \Delta : \text{Set}_n \text{ where } \overline{con}$	datatype declaration
	$ \text{record } self : R \Delta : \text{Set}_n \text{ where } \overline{field}$	record declaration
	$ \text{definition } f : A \text{ where } \overline{cls^s}$	function declaration
con	$::= \text{c } \Delta [\bar{i} \mid b]$	constructor declaration
b	$::= \epsilon \mid (u_0, u_1) b$	boundary terms
$field$	$::= \pi : A$	field declaration
$cls^\ominus$	$::= \bar{q} \hookrightarrow rhs$	unchecked clause
$cls^\oplus$	$::= \Delta \vdash \bar{q} \hookrightarrow u : B$	checked clause
rhs	$::= u$	clause body: expression
	$ \text{impossible}$	empty body for absurd pattern
Σ	$::= \overline{decl^\oplus}$	signature

Fig. 2. Declarations.

The theory is parameterized by a list of declarations Σ whose grammar is shown in Figure 2. Declaration forms are due to Cockx & Abel (2018) except for datatype constructors $\text{c } \Delta [\bar{i} \mid b]$. These take a telescope of arguments Δ , that is, a list of variable typings $(x : A)$, but now also a *boundary* $[\bar{i} \mid b]$, which specifies the dimensions $\bar{i}$ and endpoints b of path constructors. For example, `posneg` from Section 2.4.1 would be specified by `posneg  $\in [i \mid (\text{pos } 0, \text{neg } 0)]$` . We will write $\boxed{\Delta[\bar{i} \mid b] \rightarrow A}$ for the iterated function and path type defined by the following equations:

$$\begin{aligned}
[\] &\rightarrow A = A \\
[i \bar{i} \mid (u_0, u_1) b] &\rightarrow A = \text{PathP } (\lambda i. [\bar{i} \mid b] \rightarrow A) (\lambda \bar{i}. u_0) (\lambda \bar{i}. u_1) \\
(x : B) \Delta [\bar{i} \mid b] &\rightarrow A = (x : B) \rightarrow \Delta [\bar{i} \mid b] \rightarrow A
\end{aligned}$$

Further, we write $\hat{\Delta}[\bar{i} \mid b]$ for the appropriate sequence of function and path applications. A constructor $\text{c } \Delta' [\bar{i} \mid b]$ for a datatype $D \Delta$ will then have type $\Delta \Delta' [\bar{i} \mid b] \rightarrow D \hat{\Delta}$.

The core definition of Cockx & Abel (2018) is the elaboration judgment for function definitions $\Sigma; \Gamma \vdash P \mid f \bar{q} := Q : C \rightsquigarrow \Sigma'$ which performs type and coverage checking for the user-supplied clauses of f given to the judgment as P . It further computes the corresponding case tree Q and checked clauses $cls^\oplus$ in Σ' . Case trees Q are previously specified by a typing judgment $\Sigma; \Gamma \vdash f \bar{q} := Q : C \rightsquigarrow \Sigma'$ which follows the same structure but takes the case tree as input. Whenever the elaboration judgment succeeds, the typing judgment will also hold. In particular, given a signature Σ , a function declaration `definition  $f : C \text{ where } \bar{q}' \hookrightarrow rhs$` is elaborated by a call to the elaboration judgment where $\Gamma = \epsilon$, $\bar{q} = \epsilon$ and $P = \{\bar{q}'_i \hookrightarrow rhs_i \mid i = 1 \dots k\}$. In the following, we only present the rules for the case tree typing judgment and refer to the Supplemental Material for the elaboration judgment.

4.3 Case trees

Figures 3 and 4 describe the case tree typing judgment $\boxed{\Sigma; \Gamma \vdash f \bar{q} := Q : C \mid \Theta \rightsquigarrow \Sigma'}$. In our version, the judgment takes an extra input Θ which is a possibly empty list of *boundary assignments* α , which in turn are lists of assignments of interval variables to either **i0** or **i1**. We denote with $\boxed{[\alpha]}$ the substitution implied by the equalities in α itself. The list Θ is used to keep track of which faces of the current definition have accumulated some boundary constraints, due to the rules to introduce a path (CTINTROPATH), a partial element (CTSPLITPARTIAL), or to pattern-match on an higher inductive type (CTSPLITCONHIT).

In the following, we comment on the individual rules of Figure 3:

CTDONE A leaf of a case tree consists of a term v of the expected type C . Moreover, v has to fulfill the boundary constraints on the faces specified by Θ : for every α_i , we require that $f \bar{q}$ and v agree when substituted with $[\alpha_i]$, that is, when restricted to the face in question. Note that we impose the boundary constraints in the signature Σ where we have not added the clause $f \bar{q} \hookrightarrow v$ yet, so they are nontrivial to satisfy. We use $\boxed{\Gamma_\alpha}$ to denote the context obtained by removing the variables in α from Γ and substituting their occurrences with the specified values.

CTINTROPATH If the expected type C is a path type **PathP** $B u_0 u_1$, then we can extend the left-hand side to $f \bar{q} @_{u_0, u_1} i$. We also extend the list of boundary assignments to include the two faces ($i = \mathbf{i0}$) and ($i = \mathbf{i1}$), which will in CTDONE ensure that Q produces an element that connects u_0 and u_1 . To see this, note that u_0 is judgmentally equal to expression $f \bar{q} @_{u_0, u_1} \mathbf{i0}$ and u_1 to $f \bar{q} @_{u_0, u_1} \mathbf{i1}$ because of equality for path applications.

CTSPLITPARTIAL If the expected type is equal to **PartialP** $r A$,⁸ then we can proceed by splitting on the faces $\alpha_1, \dots, \alpha_n$ as long as they together *cover* all the ways in which we can have $r = \mathbf{i1}$. This is ensured by the premise $r = \bigvee_i \bigwedge \alpha_i$, where $\boxed{\bigwedge \alpha}$ is defined by mapping ($i = \mathbf{i1}$) to i and ($i = \mathbf{i0}$) to $\sim i$, and combining the resulting elements with $\bigwedge$. For each face, the left-hand side is first refined to $f \bar{q}[\alpha_i]$ so that the variables in the copatterns $\bar{q}$ match their assignment and then extended by $\boxed{[1=1]}$ which is the right elimination for **PartialP** $r A$ as we have $r = \mathbf{i1}$ in Γ_{α_i} . Additionally, since the faces α_i can have overlaps, we need to make sure that the different case trees Q_i agree on the intersections, and this is accomplished by extending Θ with the assignments of the previous cases. Finally, when substituting into a list of assignments, as in $\boxed{\Theta[\alpha]}$, any trivial equalities get removed and any contradictory ones cause the whole assignment in which they appear to be removed.

CTSPLITCONHIT If the left-hand side $f \bar{q}$ contains a variable x of a datatype **D** $\bar{v}$, then we can pattern-match on x , building a **case** _{x} {...} node that covers all the alternatives. Note that here, as well as CTSPLITCON, the datatype is not an indexed family, so we do not require a clause for **transpX**. In this rule, we deal with the case in which **D** $\bar{v}$ is an HIT, as at least one of the c_i has a non-empty boundary. For each of the constructors c_i , we check the case tree Q_i . To do so, we (1) refine $f \bar{q}$ by replacing x with c_i fully applied to variable and path application copatterns according to its type and (2) expand the list of assignments with both ($j = \mathbf{i0}$) and ($j = \mathbf{i1}$) for each

⁸ **PartialP** is a dependent version of **Partial** where A is a partial element as well, that is, $A : \mathbf{Partial} \ r \ \mathbf{Set}_n$.

$\boxed{\Sigma; \Gamma \vdash \mathbf{f} \bar{q} := Q : C \mid \Theta \rightsquigarrow \Sigma'}$ Presupposes: $\Sigma; \Gamma \vdash \mathbf{f} [\bar{q}] : C$ and $\text{dom}(\Gamma) = \text{PV}(\bar{q})$ and $\Theta = \alpha_1; \dots; \alpha_n$ where $\alpha ::= \epsilon \mid (i = \mathbf{i0}) \alpha \mid (i = \mathbf{i1}) \alpha$ such that $\Sigma; \Gamma \vdash i : \mathbb{I}$.
Checks case tree Q and outputs an extension Σ' of Σ by the clauses represented by “ $\mathbf{f} \bar{q} \hookrightarrow Q$ ”.

$$\begin{array}{c}
\frac{\Sigma; \Gamma \vdash v : C \quad \Theta = \alpha_1; \dots; \alpha_n \quad (\Sigma; \Gamma_{\alpha_i} \vdash \mathbf{f} \bar{q}[\alpha_i] = v[\alpha_i] : C[\alpha_i])_{i=1\dots n}}{\Sigma; \Gamma \vdash \mathbf{f} \bar{q} := v : C \mid \Theta \rightsquigarrow \Sigma, (\text{clause } \Gamma \vdash \mathbf{f} \bar{q} \hookrightarrow v : C)} \text{CTDONE} \\
\\
\frac{\Sigma; \Gamma \vdash C = (x : A) \rightarrow B : \text{Set}_\ell \quad \Sigma; \Gamma(x : A) \vdash \mathbf{f} \bar{q} x := Q : B \mid \Theta \rightsquigarrow \Sigma'}{\Sigma; \Gamma \vdash \mathbf{f} \bar{q} := \lambda x. Q : C \mid \Theta \rightsquigarrow \Sigma'} \text{CTINTRO} \\
\\
\frac{\Sigma; \Gamma \vdash C = \text{PathP } B u_0 u_1 : \text{Set}_n \quad \Theta' = (i = 0); (i = 1); \Theta \quad \Sigma; \Gamma(i : \mathbb{I}) \vdash \mathbf{f} \bar{q} @_{u_0, u_1} i := Q : B i \mid \Theta' \rightsquigarrow \Sigma'}{\Sigma; \Gamma \vdash \mathbf{f} \bar{q} := \lambda i. Q : C \mid \Theta \rightsquigarrow \Sigma'} \text{CTINTROPATH} \\
\\
\frac{\Sigma_0; \Gamma \vdash C = \text{PartialP } r A : \text{Set}_\omega \quad \Sigma_0; \Gamma \vdash r = \bigvee_i \bigwedge \alpha_i : \mathbb{I} \quad \left(\begin{array}{c} \Theta'_i = (\alpha_1; \dots; \alpha_{i-1}; \Theta)[\alpha_i] \\ \Sigma_{i-1}; \Gamma_{\alpha_i} \vdash (\mathbf{f} \bar{q}[\alpha_i] [\mathbf{1} = \mathbf{1}]) := Q_i : (A \mathbf{1} = \mathbf{1}) \mid \Theta'_i \rightsquigarrow \Sigma_i \end{array} \right)_{i=1\dots n}}{\Sigma_0; \Gamma \vdash \mathbf{f} \bar{q} := \text{split}\{\alpha_1 \mapsto Q_1; \dots; \alpha_n \mapsto Q_n\} : C \mid \Theta \rightsquigarrow \Sigma_n} \text{CTSPLITPARTIAL} \\
\\
\frac{\Sigma_0; \Gamma \vdash C = \mathbf{R} \bar{v} : \text{Set}_n \quad \text{record self} : \mathbf{R} \Delta : \text{Set}_n \text{ where } \overline{\pi_i : A_i} \in \Sigma_0 \quad \sigma = [\bar{v} / \Delta, \mathbf{f} [\bar{q}] / \text{self}] \quad (\Sigma_{i-1}; \Gamma \vdash \mathbf{f} \bar{q} . \pi_i := Q_i : A_i \sigma \mid \Theta \rightsquigarrow \Sigma_i)_{i=1\dots n}}{\Sigma_0; \Gamma \vdash \mathbf{f} \bar{q} := \text{record}\{\pi_1 \mapsto Q_1; \dots; \pi_n \mapsto Q_n\} : C \mid \Theta \rightsquigarrow \Sigma_n} \text{CTCOSPLIT} \\
\\
\frac{\Sigma_0; \Gamma_1 \vdash A = \mathbf{D} \bar{v} : \text{Set}_n \quad \text{data } \mathbf{D} \Delta : \text{Set}_n \text{ where } \overline{\mathbf{c}_i \Delta_i} \in \Sigma_0 \quad (\Delta'_i = \Delta_i[\bar{v} / \Delta])_{i=1\dots n} \quad (\rho_i = \mathbb{1}_{\Gamma_1} \uplus [\mathbf{c}_i \hat{\Delta}'_i / x] \quad \rho'_i = \rho_i \uplus \mathbb{1}_{\Gamma_2})_{i=1\dots n} \quad (\Sigma_{i-1}; \Gamma_1 \Delta'_i(\Gamma_2 \rho_i) \vdash \mathbf{f} \bar{q} \rho'_i := Q_i : C \rho'_i \mid \Theta \rightsquigarrow \Sigma_i)_{i=1\dots n}}{\Sigma_0; \Gamma_1(x : A) \Gamma_2 \vdash \mathbf{f} \bar{q} := \text{case}_x\{\mathbf{c}_1 \hat{\Delta}'_1 \mapsto Q_1; \dots; \mathbf{c}_n \hat{\Delta}'_n \mapsto Q_n\} : C \mid \Theta \rightsquigarrow \Sigma_n} \text{CTSPLITCON} \\
\\
\frac{\Sigma_0; \Gamma_1 \vdash A = \mathbf{D} \bar{v} : \text{Set}_n \quad \Gamma = \Gamma_1(x : A) \Gamma_2 \quad \text{data } \mathbf{D} \Delta : \text{Set}_n \text{ where } \overline{\mathbf{c}_i \Delta_i [\bar{j}_i \mid b_i]} \in \Sigma_0 \quad \exists k. [\bar{j}_k \mid b_k] \neq []}{\left(\begin{array}{c} \Delta'_i = \Delta_i[\bar{j}_i : \mathbb{I}][\bar{v} / \Delta] \quad \bar{q}_i = \hat{\Delta}_i[\bar{j}_i \mid b_i][\bar{v} / \Delta] \\ \rho_i = \mathbb{1}_{\Gamma_1} \uplus [\mathbf{c}_i \bar{q}_i / x] \quad \rho'_i = \rho_i \uplus \mathbb{1}_{\Gamma_2} \\ \Theta_i = \text{BOUNDARY}(\bar{j}_i); \Theta \\ \Sigma_{i-1}; \Gamma_1 \Delta'_i(\Gamma_2 \rho_i) \vdash \mathbf{f} \bar{q} \rho'_i := Q_i : C \rho'_i \mid \Theta_i \rightsquigarrow \Sigma_i \end{array} \right)_{i=1\dots n}}{\Sigma_n; \Gamma_1(x : \mathbf{D} \bar{v}) \Gamma_2 \vdash \mathbf{f} \bar{q} := \text{case}_x\{\text{hcomp } r u u_0 \mapsto Q_{\text{hc}}\} : C \mid \Theta \rightsquigarrow \Sigma_{n+1}} \text{CTSPLITCONHIT} \\
\\
\frac{\Sigma_n; \Gamma \vdash \mathbf{f} \bar{q} := \text{case}_x\left\{ \begin{array}{l} \mathbf{c}_1 \bar{q}_1 \mapsto Q_1; \dots; \mathbf{c}_n \bar{q}_n \mapsto Q_n \\ \text{hcomp } r u u_0 \mapsto Q_{\text{hc}} \end{array} \right\} : C \mid \Theta \rightsquigarrow \Sigma_{n+1}}{}
\end{array}$$

Fig. 3. Typing rules for case trees (excluding Eq).

$$\begin{array}{c}
\Sigma; \Gamma_1 \vdash A = \text{Eq}_B u v : \text{Set}_\ell \quad \Sigma; \Gamma_1 \vdash_p^r u =^? v : B \Rightarrow \text{YES}(\Gamma'_1, \rho, \tau, _) \\
\rho' = (\rho \uplus [\text{refl} / x]) \uplus \mathbb{1}_{\Gamma_2} \quad \tau' = \tau \uplus \mathbb{1}_{\Gamma_2} \\
\Sigma; \Gamma'_1(\Gamma_2(\rho \uplus [\text{refl} / x])) \vdash \text{f}\bar{q}\rho' := Q_{\text{refl}} : C\rho' \rightsquigarrow \Sigma_1 \\
\Sigma_1; \Gamma_1(x : \text{Eq}_B u v)\Gamma_2 \vdash \text{f}\bar{q} := \text{case}_x\{\text{hcomp } r t t_0 \mapsto Q_{\text{hc}}\} : C \rightsquigarrow \Sigma_2 \\
\Delta_{\text{tX}} = (b : B)(r : \mathbb{I})(p : \text{Path}^r B b v)(t_0 : \text{Eq}_B u b) \\
\rho_{\text{tX}} = \mathbb{1}_{\Gamma_1} \uplus [\text{transpX } p r t_0 / x] \quad \rho'_{\text{tX}} = \rho_{\text{tX}} \uplus \mathbb{1}_{\Gamma_2} \\
\Sigma_2; \Gamma_1 \Delta_{\text{tX}}(\Gamma_2 \rho_{\text{tX}}) \vdash \text{f}\bar{q}\rho'_{\text{tX}} := Q_{\text{tX}} : C\rho'_{\text{tX}} \mid (r = \text{i1}); \Theta \rightsquigarrow \Sigma' \\
\hline
\Sigma; \Gamma_1(x : A)\Gamma_2 \vdash \text{f}\bar{q} := \text{case}_x \left\{ \begin{array}{l} \text{refl} \mapsto \tau' Q_{\text{refl}} \\ \text{hcomp } r t t_0 \mapsto Q_{\text{hc}} \\ \text{transpX}_b p r t_0 \mapsto Q_{\text{tX}} \end{array} \right\} : C \rightsquigarrow \Sigma' \quad \text{CTSPLITEQ} \\
\\
\Sigma; \Gamma_1 \vdash A = \text{Eq}_B u v : \text{Set}_\ell \quad \Sigma; \Gamma_1 \vdash_p^r u =^? v : B \Rightarrow \text{NO} \\
\hline
\Sigma; \Gamma_1(x : A)\Gamma_2 \vdash \text{f}\bar{q} := \text{case}_x\{\} : C \rightsquigarrow \Sigma \quad \text{CTSPLITABSURDEQ}
\end{array}$$

Fig. 4. Typing rules for case trees involving Eq .

$\boxed{\Sigma; \Gamma \vdash \text{f}\bar{q} := \text{case}_x\{\text{hcomp } r u u_0 \mapsto Q\} : C \mid \Theta \rightsquigarrow \Sigma'}$ Presupposes: $(x : A) \in \Gamma$ where A is a type supporting hcomp , and the presuppositions made by the typing of case trees judgment (Figure 3).

Checks case tree Q can be used for the hcomp case of a split on x .

$$\begin{array}{c}
\Delta_{\text{hc}} = (r : \mathbb{I})(u : \mathbb{I} \rightarrow \text{Partial } r A)(u_0 : A [r \mapsto u \text{i0}]) \\
\rho_{\text{hc}} = \mathbb{1}_{\Gamma_1} \uplus [\text{hcomp } r u u_0 / x] \quad \rho'_{\text{hc}} = \rho_{\text{hc}} \uplus \mathbb{1}_{\Gamma_2} \\
\Sigma; \Gamma_1 \Delta_{\text{hc}}(\Gamma_2 \rho_{\text{hc}}) \vdash \text{f}\bar{q}\rho'_{\text{hc}} := Q : C\rho'_{\text{hc}} \mid (r = \text{i1}); \Theta \rightsquigarrow \Sigma' \\
\hline
\Sigma; \Gamma_1(x : A)\Gamma_2 \vdash \text{f}\bar{q} := \text{case}_x\{\text{hcomp } r u u_0 \mapsto Q\} : C \mid \Theta \rightsquigarrow \Sigma'
\end{array}$$

Fig. 5. Typing a match against hcomp .

j in $\bar{j}_i$, which is what $\boxed{\text{BOUNDARY}(\bar{j}_i)}$ denotes. Moreover, we need to consider the case for $\text{hcomp } r u u_0$ (see Figure 5): we check Q_{hc} by (1) replacing x by the pattern $\text{hcomp } r u u_0$ in the left-hand side and (2) extending Θ with $(r = \text{i1})$ since that is the face where $\text{hcomp } r u u_0$ computes to $u \text{i1} \mathbf{1} = \mathbf{1}$.

We discuss rule CTSPLITEQ in the next section, while the remaining rules do not directly involve any of the new features of Cubical Agda, so we ask the reader to refer back to the corresponding judgment in Cockx & Abel (2018).

4.3.1 Unification: Splitting on the inductive equality type

The rule CTSPLITEQ allows splitting on $\text{Eq}_B u v$ only when u and v can be unified by some substitution ρ , so that refl will be typeable at $\text{Eq}_{B\rho} u\rho v\rho$. Cockx & Devries (2018)

define a proof-relevant notion of unification, where most general unifiers are equivalences of the form $\Gamma(x : \text{Id}_B u v) \simeq \Gamma'$. Moreover, they also define a notion of *strong unifier* which requires additional definitional equalities to be satisfied by the equivalence, and these guarantee that reductions are preserved when a case tree is translated to eliminators. Cockx & Abel (2018) adapt the notion of strong unifier to their specific setting, but they only preserve the substitutions between the two contexts, as the proofs are guaranteed to specialize to reflexivity when the element of $\text{Eq}_B u v$ is **refl**, and they have no other canonical elements of that type. In our context, the arguments r and p of $\text{transpX} p r t$, when paired together, correspond to the canonical elements of Swan's Id type (Cohen et al., 2018), which supports the same interface of the identity type used in Cockx & Devriese (2018) to define their unifiers, as shown in Section 9.1 of Cohen et al. (2018). Accordingly, we define $\boxed{\text{Path}^r A a_0 a_1}$ to be the type of paths $a_0 \equiv_A a_1$ that are **refl** when $r = \text{i1}$, so that we can use a telescope like $(r : \mathbb{I})(p : \text{Path}^r A a_0 a_1)$ to represent unification problems, and the inputs to transpX . The notion of $\text{Path}^r A a_0 a_1$ has an extensional flavor because an assumption $(p : \text{Path}^{\text{i1}} A a_0 a_1)$ implies the definitional equality of a_0 and a_1 . For this reason, we only introduce it as a bookkeeping notation in the typing context of case trees and merely regard it as an aid to express the meta-theory developed here. To keep track of paths between substitutions, we define a *substitution path* between substitutions $\Gamma \vdash \sigma_0, \sigma_1 : \Delta$ to be a substitution $\Gamma(i : \mathbb{I}) \vdash \eta : \Delta$ such that $\Gamma \vdash \eta[\text{i0}/i] = \sigma_0 : \Delta$ and $\Gamma \vdash \eta[\text{i1}/i] = \sigma_1 : \Delta$. We also say that a substitution path $\Gamma(i : \mathbb{I}) \vdash \eta : \Gamma$ is *constant* if it is equal to $\mathbb{1}_\Gamma$ weakened by $(i : \mathbb{I})$. We now have everything in place to give our definitions of strong unifier and disunifier.

Definition 1 (Strong unifier). Let Γ be a well-formed context and u and v be terms such that $\Gamma \vdash u, v : A$. A strong unifier $(\Gamma', \sigma, \tau, \eta)$ of u and v consists of a context Γ' and substitutions $\Gamma' \vdash \sigma : \Gamma(r : \mathbb{I})(p : \text{Path}^r A u v)$ and $\Gamma(r : \mathbb{I})(p : \text{Path}^r A u v) \vdash \tau : \Gamma'$ and a substitution path η between $\sigma; \tau$ and $\mathbb{1}_{\Gamma(r : \mathbb{I})(p : \text{Path}^r A u v)}$ such that:

1. $\Gamma' \vdash r\sigma = \text{i1} : \mathbb{I}$ and $\Gamma' \vdash p\sigma = \text{refl} : \text{Path}^{\text{i1}} A \sigma u \sigma v \sigma$ (these imply the definitional equality $\Gamma' \vdash u\sigma = v\sigma : A\sigma$).
2. $\Gamma' \vdash \tau; \sigma = \mathbb{1}_{\Gamma'} : \Gamma'$
3. For any $\Gamma_0 \vdash \sigma_0 : \Gamma(r : \mathbb{I})(p : \text{Path}^r A u v)$ such that $\Gamma_0 \vdash r\sigma_0 = \text{i1} : \mathbb{I}$ and $\Gamma_0 \vdash p\sigma_0 = \text{refl} : \text{Path}^{\text{i1}} A \sigma_0 u \sigma_0 v \sigma_0$, we have that $\Gamma_0 \vdash \sigma; \tau; \sigma_0 = \sigma_0 : \Gamma(r : \mathbb{I})(p : \text{Path}^r A u v)$ and that $\eta; (\sigma_0 \uplus \mathbb{1}_{i : \mathbb{I}})$ is a constant substitution path.

The last condition about $\eta; (\sigma_0 \uplus \mathbb{1}_{i : \mathbb{I}})$ being a constant substitution path makes sure that transporting along η will be the identity whenever r and p are solved by **i1** and **refl**. It corresponds to the analogous condition about isLinv in Definition 53 of Cockx & Devriese (2018). Note that while η is not used in the CTSPLITEQ typing rule, it will be necessary in Section 4.3.3.

Definition 2 (Disunifier). Let Γ be a well-formed context and $\Gamma \vdash u, v : A$. A disunifier of u and v is a function $\Gamma \vdash f : (u \equiv_A v) \rightarrow \perp$ where $\perp$ is the empty type.

Finally, we can assume the existence of a proof relevant unification algorithm which we specify through the following judgments:

- A positive success $\Sigma; \Gamma \vdash_p^r u =^? v : B \Rightarrow \text{YES}(\Gamma', \rho, \tau, \eta)$ ensures that the tuple $(\Gamma', \rho \uplus [\text{il} / r, \text{refl} / p], \tau, \eta)$ is a strong unifier.
- A negative success $\Sigma; \Gamma \vdash_p^r u =^? v : B \Rightarrow \text{NO}$ ensures that there exist a disunifier of u and v .

Note that ρ is a *pattern substitution*, that is, it contains only variables and forced arguments, so that it can be applied to patterns. In general, the algorithm might also fail to provide a definitive answer, in which case no split on $\text{Eq}_B u v$ is allowed.

Remark 3. Note that the unification rules from Cockx & Devriese (2018) that are specific to datatypes will not apply to datatypes with path constructors, as properties like injectivity and distinctness of constructors cannot be guaranteed to hold in that case. In principle, we could ask the user to provide suitable proofs of these properties, but there is no such interface at the moment.

4.3.2 Inferring right-hand side of a $\text{hcomp } r u u_0$ match

In the examples given in Section 2, we have never given a clause for the hcomp constructor when pattern matching on an element of a higher inductive type. That is because Cubical Agda generates a suitable clause for us during elaboration. How to deal with the hcomp case was already explained in Cohen *et al.* (2018), but only for the respective induction principle, while in our case we have to deal with user clauses that include multiple arguments and nested pattern matching.

Fortunately in the context of the rule CTSPLITCONHIT , we have the right information available to construct a term that would be suitable for Q_{hc} . This is accomplished in Figure 6 by the only rule of the judgment $\Sigma; \Gamma(x : D \bar{v}) \Delta \vdash \bar{f} \bar{q} : C \mid \Theta \Rightarrow^x \text{HC-RHS}(rhs)$. The term rhs is supposed to be typable as:

$$\Gamma \Delta_{\text{hc}} \Delta[\text{hcomp } u u_0 / x] \vdash rhs : C[\text{hcomp } u u_0 / x]$$

while also satisfying the constraints implied by $r = \text{il}; \Theta$. The construction of rhs is fairly involved, so we will build up to it with simpler cases.

First, let us assume both Δ and Θ are empty and that $C = T$ where T does not depend on x , which means we want $\Gamma \Delta_{\text{hc}} \vdash rhs : T$. We already have $\Gamma(x : D \bar{v}) \vdash \bar{f} \bar{q} : T$, which we can use with x replaced by elements of type $D \bar{v}$ obtained by u and u_0 and build a composition in T . Writing $g(d)$ for $\bar{f} \bar{q}[d / x]$, we define rhs as:

$$\Gamma \Delta_{\text{hc}} \vdash rhs := \text{hcomp } (\lambda\{i (r = \text{il}) \rightarrow g(u i \text{1=1})\}) (\text{inS}(g(\text{outS } u_0))) : T$$

Note that $\Gamma \Delta_{\text{hc}}, r = \text{il} \vdash rhs = \bar{f} \bar{q}[u i \text{1=1} / x]$ as expected, while defining rhs as just $g(\text{outS } u_0)$ would not have satisfied this equality.

Slightly more complex is the case where T does depend on x , so that $g(u i \text{1=1})$ has type $T[u i \text{1=1} / x]$ and $g(\text{outS } u_0)$ has type $T[\text{outS } u_0 / x]$, while rhs will have type $T[\text{hcomp } u u_0 / x]$. What we need then is to use heterogeneous composition, comp , along with a suitable family $A : \mathbb{I} \rightarrow \text{Set}_\ell$ which matches the three types above in the respective

$$\boxed{\Sigma; \Gamma(x : \mathbb{D} \bar{v}) \Delta \vdash \mathbf{f} \bar{q} : C \mid \Theta \Rightarrow^x \text{HC-RHS}(rhs)}$$

$$\begin{aligned}
\Theta &= \alpha_1; \dots; \alpha_n \quad \Delta_{\text{hc}} = (r : \mathbb{I})(u : \mathbb{I} \rightarrow \text{Partial } r(\mathbb{D} \bar{v}))(u_0 : \mathbb{D} \bar{v} [r \mapsto u \mathbf{i0}]) \\
\Delta^i &= \Delta[\mathbf{hfill} \ u \ u_0 \ i / x] \\
\delta^i : \Delta^i &= \text{TRANSP-TEL} (j. \Delta^{\sim j \vee i}) i \hat{\Delta}^{\mathbf{i1}} \\
sys &= \lambda i. \left\{ \begin{array}{ll} (r = \mathbf{i1}) \rightarrow \mathbf{f} \bar{q}[u \ i \ \mathbf{1=1} / x, \delta^i / \Delta^i] \\ \alpha_1 & \rightarrow \mathbf{f} \bar{q}[\mathbf{hfill} \ u \ u_0 \ i / x, \delta^i / \Delta^i][\alpha_1] \\ \vdots & \\ \alpha_n & \rightarrow \mathbf{f} \bar{q}[\mathbf{hfill} \ u \ u_0 \ i / x, \delta^i / \Delta^i][\alpha_n] \end{array} \right\} \\
rhs &= \text{comp} (\lambda i. C[\mathbf{hfill} \ u \ u_0 \ i / x, \delta^i / \Delta^i]) sys (\text{inS} (\mathbf{f} \bar{q}[\text{outS} \ u_0 / x, \delta^{\mathbf{i0}} / \Delta])) \\
\text{Derivable typing:} \quad & \Gamma \Delta_{\text{hc}} \Delta[\mathbf{hcomp} \ u \ u_0 / x] \vdash rhs : C[\mathbf{hcomp} \ u \ u_0 / x]
\end{aligned}$$

$$\Sigma; \Gamma(x : \mathbb{D} \bar{v}) \Delta \vdash \mathbf{f} \bar{q} : C \mid \Theta \Rightarrow^x \text{HC-RHS}(rhs)$$

Fig. 6. Computing the right-hand side of a **hcomp** match.

cases. We have already seen that **hfill** $u \ u_0$ is a filler for the open box specified by u and u_0 and with lid **hcomp** $u \ u_0$, so taking $A = \lambda i. T[\mathbf{hfill} \ u \ u_0 \ i / x]$ will do the job:

$$\Gamma \Delta_{\text{hc}} \vdash rhs := \text{comp } A (\lambda \{i (r = \mathbf{i1}) \rightarrow g(u \ i \ \mathbf{1=1})\}) (\text{inS}(g(\text{outS} \ u_0))) : T[\mathbf{hcomp} \ u \ u_0 / x]$$

this definition also matches the behavior of the eliminator for spheres in Section 9.2 of Cohen *et al.* (2018).

Let us now consider that we have a single y of type $Y : \text{Set}_\ell$ in Δ :

$$\Gamma \Delta_{\text{hc}}(y : Y[\mathbf{hcomp} \ u \ u_0 / x]) \vdash rhs : T[\mathbf{hcomp} \ u \ u_0 / x]$$

This case could be seen as a special case of the previous one, by replacing T with $(y : Y) \rightarrow T$ and having $\lambda y. g(d)$ in place of $g(d)$. However, for the sake of consistency with other cases, we write out explicitly how such a composition can be obtained. Let $g(d, v) = \mathbf{f} \bar{q}[d / x, v / y]$ and $y^i = \text{transp} (\lambda j. Y[\mathbf{hfill} \ u \ u_0 (\sim j \vee i) / x]) i y$ and $A = \lambda i. T[\mathbf{hfill} \ u \ u_0 \ i / x, y^i / y]$, then

$$rhs = \text{comp } A (\lambda \{i (r = \mathbf{i1}) \rightarrow g(u \ i \ \mathbf{1=1}, y^i)\}) (\text{inS}(g(\text{outS} \ u_0), y^{\mathbf{i0}})) : T[\mathbf{hcomp} \ u \ u_0 / x].$$

Compare this composition with the definition of **transportPi** in Section 3.2.1.

Alternatively, Δ could be $(k : \mathbb{I})$, with $\Theta = (k = \mathbf{i0})$; $(k = \mathbf{i1})$, as might happen when $\bar{q}$ contains a path application. Then rhs should have typing

$$\Gamma \Delta_{\text{hc}}(k : \mathbb{I}) \vdash rhs : T[\mathbf{hcomp} \ u \ u_0 / x]$$

and be equal to $\mathbf{f} \bar{q}[\mathbf{hcomp} \ u \ u_0 / x, \mathbf{i0} / k]$ whenever $k = \mathbf{i0}$, and likewise for $\mathbf{i1}$. As in the implementation of **transportPath**, we can address these constraints by adding to the sides of the composition. Let $g(d, s) = \mathbf{f} \bar{q}[d / x, s / k]$, and A be as in the empty Δ case, then we define rhs as:

$$\begin{aligned}
sys &= \lambda i. \left\{ \begin{array}{l} (r = \mathbf{i1}) \rightarrow g(u \ i \ \mathbf{1=1}, k) \\ (k = \mathbf{i0}) \rightarrow g(\mathbf{hcomp} \ u \ u_0, \mathbf{i0}) \\ (k = \mathbf{i1}) \rightarrow g(\mathbf{hcomp} \ u \ u_0, \mathbf{i1}) \end{array} \right\} \\
rhs &= \text{comp } A sys (\text{inS}(g(\text{outS} \ u_0, k))) : T[\mathbf{hcomp} \ u \ u_0 / x]
\end{aligned}$$

Note that it does not matter from which part of the context k comes from for the definition above to be well typed, so the same strategy applies also when Θ refers to variables in Γ .

Finally, when we let Δ and Θ contain multiple variables and equations, we end up with the definition given in Figure 6. There we use a version of `transp` generalized to telescopes, `TRANSP-TEL`, which covers both the cases like $(k : \mathbb{I})$ where nothing is to be done, and $(\gamma : Y)$ where `transp` for Y is used. It can also fail if the type does not support `transp` but it is not closed, in which case elaboration fails.

During elaboration, we can then run the algorithm expressed by this judgment and generate internally a clause for `hcomp` $r u u_0$.

4.3.3 Inferring case tree of a `transpX` $p r t_0$ match

What we discussed in the previous section about a `hcomp` match also holds if we are splitting on an inductive family like `EqB` $u v$. Additionally, however, we have to handle the possibility that we are matching against an element built with `transpX` $p r t_0$. In this case, we will produce a case tree that performs a further split on t_0 , so that we have to produce right-hand sides that fit the following clauses for the function f :

$$\begin{aligned} f \bar{q}[\text{transpX } p r \text{ refl}/x] &= rhs_{\text{refl}} \\ f \bar{q}[\text{transpX } p r (\text{transpX } q s t_1)/x] &= rhs_{\text{tX}} \\ f \bar{q}[\text{transpX } p r (\text{hcomp } s w w_0)/x] &= rhs_{\text{hc}} \end{aligned}$$

the term rhs_{hc} can be computed using the HC-RHS judgment from the previous section, we only need to specialize the copatterns $\bar{q}$ to $\bar{q}[\text{transpX } p r x/x]$ and update the other arguments accordingly. Specifically, using the definitions from rule CTSPLITEQ,

$$\Sigma; \Gamma_1 \Delta_{\text{tX}} \Gamma_2 \rho_{\text{tX}} \vdash f \bar{q} \rho'_{\text{tX}} : C \rho'_{\text{tX}} \mid (r = \text{i1}); \Theta \Rightarrow^{t_0} \text{HC-RHS}(rhs_{\text{hc}}).$$

The other two terms, rhs_{refl} and rhs_{tX} , are obtained using the judgments in Figures 7 and 8.

The judgment $\Sigma; \Gamma(x : \text{Eq}_A u v) \Delta \mid (\Gamma', \rho, \tau, \eta) \vdash f \bar{q} : C \mid \Theta \Rightarrow \text{TRXREFL}(rhs)$ in Figure 7 is where we make use of the η component of the strong unifier obtained from the judgment $\Sigma; \Gamma \vdash_p^r u =^? v : A \Rightarrow \text{YES}(\Gamma', \rho, \tau, \eta)$. Let us focus on the case where Δ and Θ are empty, then we want to construct a term rhs with typing

$$\Gamma(r : \mathbb{I})(p : \text{Path}^r u v) \vdash rhs : C[\text{transpX } p r \text{ refl}/x]$$

By the definition of strong unifier, we have that $\sigma := \rho \uplus [\text{i1}/r, \text{refl}/p]$ is an equivalence between $\Gamma(r : \mathbb{I})(p : \text{Path}^r u v)$ and Γ' , so we can use it to rewrite our goal to

$$\Gamma' \vdash ?0 : C[\text{transpX } p r \text{ refl}/x] \sigma$$

By simplifying substitutions and reducing `transpX`, the type of $?0$ is equal to $C[\rho \uplus \text{refl}/x]$, which is the type of $f \bar{q}[\rho \uplus \text{refl}/x]$, so we can use that to conclude. Writing an explicit term for the reasoning above, we get

$$\begin{aligned} rhs &= \text{comp } (\lambda i. C[\text{transpX } p r \text{ refl}/x] \eta) \\ &\quad (\lambda \{i (r = \text{i1}) \rightarrow f \bar{q}[\rho \uplus \text{refl}/x] \tau\}) \\ &\quad (\text{inS}(f \bar{q}[\rho \uplus \text{refl}/x] \tau)) \end{aligned}$$

$$\boxed{\Sigma; \Gamma(x : \text{Eq}_A u v) \Delta \mid (\Gamma', \rho, \tau, \eta) \vdash \bar{f} \bar{q} : C \mid \Theta \Rightarrow \text{TRXREFL}(rhs)}$$

$$\begin{aligned}
\Theta &= \alpha_1; \dots; \alpha_n \quad \Delta_{\text{refl}} = (r : \mathbb{I})(p : \text{Path}^r B u v) \\
\Delta^i &= \Delta[\text{transpX } p \text{ } r \text{ refl} / x][\eta] \\
\delta^i : \Delta^i &= \text{TRANSP-TEL } (j. \Delta^{j \vee i}) (i \vee r) \hat{\Delta}^{\text{i1}} \\
\text{sys} &= \lambda i. \left\{ \begin{array}{ll} (r = \text{i1}) & \rightarrow \bar{f} \bar{q}[(\rho \uplus \text{refl} / x)[\tau] \uplus \delta^i / \Delta] \\ \alpha_1 & \rightarrow \bar{f} \bar{q}[\text{transpX } p \text{ } r \text{ refl} / x][\eta \uplus \delta^i / \Delta^i][\alpha_1] \\ \vdots & \\ \alpha_n & \rightarrow \bar{f} \bar{q}[\text{transpX } p \text{ } r \text{ refl} / x][\eta \uplus \delta^i / \Delta^i][\alpha_n] \end{array} \right\} \\
\text{base} &= \text{inS } (\bar{f} \bar{q}[(\rho \uplus \text{refl} / x)[\tau] \uplus \delta^{\text{i0}} / \Delta]) \\
\text{rhs} &= \text{comp } (\lambda i. C[\text{transpX } p \text{ } r \text{ refl} / x][\eta \uplus \delta^i / \Delta^i]) \text{ sys base} \\
\text{Derivable typing:} \quad & \Gamma \Delta_{\text{refl}} \Delta[\text{transpX } p \text{ } r \text{ refl} / x] \vdash \text{rhs} : C[\text{transpX } p \text{ } r \text{ refl} / x]
\end{aligned}$$

$$\Sigma; \Gamma(x : \text{Eq}_A u v) \Delta \mid (\Gamma', \rho, \tau, \eta) \vdash \bar{f} \bar{q} : C \mid \Theta \Rightarrow \text{TRXREFL}(rhs)$$

Fig. 7. Computing the right-hand side of a $\text{transpX } p \text{ } r \text{ refl}$ match.

Finally, the judgment $\Sigma; \Gamma(x : \text{Eq}_A u v) \Delta \vdash \bar{f} \bar{q} : C \mid \Theta \Rightarrow \text{TRXTRX}(rhs)$ in Figure 8 is where we take care of the case $\bar{f} \bar{q}[\text{transpX } p \text{ } r (\text{transpX } q \text{ } s \text{ } t) / x]$ by making use of the case $\bar{f} \bar{q}[\text{transpX } (q \bullet^{s,r} p) (r \wedge s) t]$, which has one fewer transpX and so gets us closer to the $\text{transpX} __\text{refl}$ base case. The expression $(q \bullet^{s,r} p)$ is built with a transitivity operator that makes use of the s and r argument to reduce to q when $r = \text{i1}$ and reduce to p when $s = \text{i1}$, making eq_0 and eq_1 definitionally equal under either condition. Using connections and transports, we can define both eq and $eq_{s,r}$ as specified and then we proceed to define the required term rhs by nesting compositions inside an homogeneous composition. The term obtained with $c(\text{i0}, eq)$ would have the right type and satisfy the boundary conditions from Θ , but it would not satisfy the ones imposed by $(s = \text{i1}); (r = \text{i1})$, the matching cases of sys' take care of that.

5 Glue types in Cubical Agda

Glue types are the key contribution of Cohen *et al.* (2018) for equipping the univalence principle with computational content. Given that a type in cubical type theory stands for a higher-dimensional cube, **Glue** types let us construct a cube where some faces have been replaced by equivalent types. This is analogous to how **hcomp** lets us replace some faces of a cube by composing it with other cubes; however for **Glue** types, we can compose with equivalences instead of paths. This implies the univalence principle and it is what lets us transport along paths built out of equivalences. **Glue** types were originally also used to implement composition for the universe Cohen *et al.* (2018); however, Cubical Agda uses a dedicate type former for this purpose, which avoids needlessly converting paths into equivalences.

5.1 Glue types and univalence

As everything in Cubical Agda has to work up to higher dimensions, the **Glue** types take a partial family of types A that are equivalent to the base type B . The idea is then that

$$\boxed{\Sigma; \Gamma(x : \mathbf{Eq}_A u v) \Delta \vdash \mathbf{f} \bar{q} : C \mid \Theta \Rightarrow \text{TRXTRX}(rhs)}$$

$$\begin{aligned} \Theta &= \alpha_1; \dots; \alpha_n \quad \Delta_{\mathbf{tX}} = (b : B)(r : \mathbb{I})(p : \mathbf{Path}^r B b v) \\ &\quad (b' : B)(s : \mathbb{I})(q : \mathbf{Path}^s B b' b)(t : \mathbf{Eq}_B u b') \\ eq_0 &= \mathbf{transpX} (q \bullet^{s,r} p) (r \wedge s) t \quad eq_1 = \mathbf{transpX} p r (\mathbf{transpX} q s t) \\ \text{Let } eq : eq_0 &\equiv eq_1 \text{ and } eq_{s,r} : \mathbf{PartialP} (s \vee r) (\lambda (s \vee r = \mathbf{i1}) \rightarrow eq \equiv \mathbf{refl}) \\ &\text{both constant when } r \wedge s = \mathbf{i1}. \\ \Delta' &= \Delta[eq_1 / x] \\ \delta(r', eq', j) &= \text{TRANSP-TEL} (i. \Delta[eq'(\sim i \vee j) / x]) ((s \wedge r) \vee r' \vee j) \hat{\Delta}' \\ f(r', eq', j) &= \mathbf{f} \bar{q}[eq'(j) / x, \delta(r', eq', j) / \Delta] \\ C'(r', eq') &= \lambda j \rightarrow C[eq'(j) / x, \delta(r', eq', j) / \Delta] \\ sys(r', eq') &= \lambda j \rightarrow \left\{ \begin{array}{l} \alpha_1 \rightarrow f(r', eq', j)[\alpha_1] \\ \vdots \\ \alpha_n \rightarrow f(r', eq', j)[\alpha_n] \\ (r \wedge s = \mathbf{i1}) \rightarrow f(r', eq', j) \\ (r' = \mathbf{i1}) \rightarrow f(r', eq', j) \end{array} \right\} \\ c(r', eq') &= \mathbf{comp} C'(r', eq') sys(r', eq') (\mathbf{inS} (f(r', eq', \mathbf{i0}))) \\ sys' &= \lambda i \rightarrow \left\{ \begin{array}{l} \alpha_1 \rightarrow \mathbf{f} \bar{q}[eq_0 / x][\alpha_1] \\ \vdots \\ \alpha_n \rightarrow \mathbf{f} \bar{q}[eq_0 / x][\alpha_n] \\ (r = \mathbf{i1}) \rightarrow c(i, eq_{s,r} \mathbf{1=1} i) \\ (s = \mathbf{i1}) \rightarrow c(i, eq_{s,r} \mathbf{1=1} i) \end{array} \right\} \\ rhs &= \mathbf{hcomp} sys' c(\mathbf{i0}, eq) \\ \text{Derivable typing:} \quad &\Gamma \Delta_{\mathbf{tX}} \Delta' \vdash rhs : C[eq_1 / x] \\ \hline &\Sigma; \Gamma(x : \mathbf{Eq}_A u v) \Delta \vdash \mathbf{f} \bar{q} : C \mid \Theta \Rightarrow \text{TRXTRX}(rhs) \end{aligned}$$

Fig. 8. Computing the right-hand side of a $\mathbf{transpX} p r (\mathbf{transpX} q s t)$ match.

these types get glued onto B , so that the equivalence data gets packaged up into a new datatype:

$$\mathbf{Glue} : (B : \mathbf{Set} \ell) \{r : \mathbb{I}\} \rightarrow \mathbf{Partial} r (\Sigma [A \in \mathbf{Set} \ell] (A \simeq B)) \rightarrow \mathbf{Set} \ell$$

When r is $\mathbf{i1}$ the type $\mathbf{Glue} \{r\} B A e$ reduces to $A e \mathbf{1=1} .\mathbf{fst}$.

Using $\mathbf{Glue}$ types, we can turn an equivalence of types into a path and hence define $\mathbf{ua}$.

$$\begin{aligned} \mathbf{ua} : \{A B : \mathbf{Set} \ell\} &\rightarrow A \simeq B \rightarrow A \equiv B \\ \mathbf{ua} \{A = A\} \{B = B\} e i &= \mathbf{Glue} B (\lambda \{ (i = \mathbf{i0}) \rightarrow (A, e) \\ &\quad ; (i = \mathbf{i1}) \rightarrow (B, \mathbf{idEquiv} B) \}) \end{aligned}$$

The idea is that we glue A onto B when i is $\mathbf{i0}$ using e and B onto itself when i is $\mathbf{i1}$ using the identity equivalence. The term $\mathbf{ua} e$ is a path from A to B as the $\mathbf{Glue}$ type reduces when

the face conditions are satisfied, so when i is **i0** this reduces to A and when i is **i1** it reduces to B . Pictorially, we can describe $\mathbf{ua} \ e$ as the dashed line in:

$$\begin{array}{ccc} A & \xrightarrow{\quad\quad\quad} & B \\ \downarrow e \wr & & \downarrow \wr \text{idEquiv } B \\ B & \xrightarrow{\quad B \quad} & B \end{array}$$

The **transp** operation for **Glue** types is the most complicated part of the internals of Cubical Agda. The algorithm closely follows Huber (2017, Section 3.6), which is a variation of the original algorithm from Cohen *et al.* (2018, Section 6.2). We will focus on the special case of **transport** $(\lambda i \rightarrow \mathbf{ua} \ e \ i) \ a$ for simplicity. This will transport a from A to B by going through the three fully filled lines in the above picture.

Unfolding **ua** gives

$$\mathbf{transport} \ (\lambda i \rightarrow \mathbf{Glue} \ B \ (\lambda \{ (i = \mathbf{i0}) \rightarrow (A, e) ; (i = \mathbf{i1}) \rightarrow (B, \text{idEquiv } B) \}) \ a)$$

By the boundary equations for **Glue** types, we get that $a : A$ (as it is in the $i = \mathbf{i0}$ face of the **Glue** type). The algorithm then applies the function of e (i.e., $e \ .\mathbf{fst} : A \rightarrow B$) to a giving an element in B . As B is constant along i we could now be done; however, for the general algorithm, there is no reason for the base to be constant along i ; it could for example be another **Glue** type! We must hence transport along $(\lambda i \rightarrow B)$ to get an element in the bottom-right B in the diagram. In order to go up to the top-right corner, we then use the inverse of the identity equivalence.⁹ Since this is the identity function, we end up with:

$$\mathbf{transport} \ (\lambda i \rightarrow B) \ (e \ .\mathbf{fst} \ a)$$

Using the same path as in the definition of **transport** for path types, we can prove that this is equal to $e \ .\mathbf{fst} \ a$ up to a path:

$$\begin{aligned} \mathbf{ua}\beta : \{A \ B : \mathbf{Set} \ \ell\} \ (e : A \simeq B) \ (a : A) &\rightarrow \mathbf{transport} \ (\mathbf{ua} \ e) \ a \equiv e \ .\mathbf{fst} \ a \\ \mathbf{ua}\beta \ \{B = B\} \ e \ a = \lambda i \rightarrow \mathbf{transp} \ (\lambda _ \rightarrow B) \ i \ (e \ .\mathbf{fst} \ a) \end{aligned}$$

Transporting along the path that we get from applying **ua** to an equivalence is, thus, the same as applying the equivalence. This makes it possible to use the univalence axiom computationally in Cubical Agda: we can package up equivalences as paths, do equality reasoning using these paths, and in the end transport along the paths to compute with the equivalences. Furthermore, the combination of **ua** and **uaβ** is sufficient to prove that **ua** is an equivalence which gives the full univalence theorem, that is, an equivalence between paths and equivalences:¹⁰

$$\mathbf{univalence} : \forall \{\ell\} \{A \ B : \mathbf{Set} \ \ell\} \rightarrow (A \equiv B) \simeq (A \simeq B)$$

5.2 General case of **transp** for **Glue** types and the **ghcomp** operation

While the special case of **transp** for **Glue** types above is quite simple, the general case is a lot more complex. The reason is that the input might depend on many more variables than just i . When considering

⁹ In general, this might not be the identity function, thus, this step might actually do something.

¹⁰ <https://github.com/agda/cubical/blob/master/Cubical/Foundations/Univalence.agda#L63>.

`transport` ($\lambda i \rightarrow \text{Glue } B (\lambda \{ (r = \mathbf{i1}) \rightarrow (A, e) \}) a$

the interval element r might be quite complex and its disjunctive normal form might contain clauses that do not involve i . On these parts, the `transp` function should compute like the `transp` function for A by the boundary rules for `Glue` types. This in turn means that additional corrections have to be made compared to the `ua` case. In Cohen *et al.* (2018), the part of r that does not mention i is written $\forall i.r$ (as this operation corresponds to universal quantification on the interval).¹¹

One of the modifications we have to do in the general case of `transp` for `Glue` types is that the simple `transport` in B has to be a `comp` with suitable corrections for the $\forall i.r$ faces. While this is easily achieved, it has some unfortunate consequences in the case of transporting along `ua`. In this particular case, r is $i \vee \sim i$ so that $(\forall i.r) = \mathbf{i0}$ as there is no part that does not mention i . This means that the `comp` correction will introduce an empty system which implies that our simple proof of `uaβ` does not work anymore. In order to fix this, we have to extend the proof of `uaβ` with a suitable `hfill` in order to compensate for the additional empty system.

Luckily, there is a simple trick in Cubical Agda that lets us adapt the correction to eliminate the empty system. The problem with the above-sketched definition is that the `comp` does not reduce when $\forall i.r$ is $\mathbf{i0}$; however, if we add a clause mapping to the base for this case, the issue with the empty system goes away. This relies on a subtle difference between the `hcomp` operation in Cubical Agda and the one in Coquand *et al.* (2018). In the latter, the boundary constraints were elements of the face lattice $\mathbb{F}$ generated by formal generators $(i = \mathbf{i0})$ and $(i = \mathbf{i1})$ subject to the relation $(i = \mathbf{i0}) \wedge (i = \mathbf{i1}) = \perp$. In Cubical Agda on the other hand, the `hcomp` operation takes a family of partial elements that are specified by some $r : \mathbb{I}$. This means that we in Cubical Agda can add a face when $(r = \mathbf{i0})$ which was not possible in Coquand *et al.* (2018) as there is no corresponding operation for $\mathbb{F}$.

The reason that $\mathbb{F}$ in Coquand *et al.* (2018) does not admit such an operation is that while every $\varphi : \mathbb{F}$ is expressible as $r = \mathbf{i1}$, the choice of r is not unique. In particular, for $\varphi = 0_{\mathbb{F}}$, we can choose either $\mathbf{i0}$ or $i \wedge \sim i$ which would give different results when equated to $\mathbf{i0}$. Using $r : \mathbb{I}$ to specify boundaries in Cubical Agda avoids the need to make such a choice, and in particular $(r = \mathbf{i0})$ is represented by $\neg r$. It would be tempting to instead extend $\mathbb{F}$ with a negation operation; however, that would allow us to represent new kinds of boundaries, like the open interval $(0, 1]$ as $\neg(i = \mathbf{i0})$, and it is not clear how they would impact decidability of type checking. Modifying `hcomp` and `transp` to take a $r : \mathbb{I}$ is semantically justified by the fact that it is not necessary for boundaries to be specified by a subobject of the subobject classifier Ω in the presheaf topos of cubical sets in order to obtain a model of univalent type theory.¹²

Inspired by Angiuli *et al.* (2017, p. 53), we call the homogeneous version of this operation *generalized homogeneous composition*, `ghcomp`. The heterogeneous version used above can be implemented using `ghcomp` in the definition of `comp`. We can write the `ghcomp` operation as:

¹¹ Technically speaking, the $\forall$ operation in Cohen *et al.* (2018) is not an operation on the interval, but rather on the face lattice $\mathbb{F}$. However, it is direct to define an analogous operation on the interval and it is this one we use here.

¹² This generalization has been formally verified in Agda in <https://github.com/mortberg/gen-cart/>.

$\text{ghcomp} : \{r : I\} (u : I \rightarrow \text{Partial } r A) (u_0 : A [r \mapsto u \text{ i0}]) \rightarrow A$
 $\text{ghcomp} \{r = r\} u u_0 =$
 $\text{hcomp} (\lambda j \rightarrow \lambda \{ (r = \text{i1}) \rightarrow u j \text{ 1=1} ; (r = \text{i0}) \rightarrow \text{outS } u_0 \})$
 $(\text{inS } (\text{outS } u_0))$

Using this in all of the places where the $\forall$ correction has to be made in the general algorithm for `transp` for `Glue`, we obtain a better algorithm which does not produce any new empty systems. This way the proof of `uaβ` can stay as simple as above and no additional corrections has to be made. This is an improvement compared to the algorithm in Cohen *et al.* (2018) (that is implemented in `cubicaltt`) which produced a surprisingly large number of empty systems even in simple cases.

6 Meta-theory of cubical type theory and Cubical Agda

The original formulation of cubical type theory as in Cohen *et al.* (2018) has a model in Kan cubical sets with connections and reversals, that is, presheaves on a suitable cube category where types have structure corresponding to the `comp` operation. This model has been formally verified in both the NuPRL proof assistant by Bickford (2018) and using Agda as the internal language of the presheaf topos of cubical sets by Orton & Pitts (2016) and Licata *et al.* (2018). This hence provides semantic consistency proofs for the cubical type theory that Cubical Agda is based on. Applying Tait’s computability method, a syntactic consistency proof for this cubical type theory was given in Huber (2016) by defining an operational semantics and proving that any term of type $\mathbb{N}$ computes to a numeral.

A crucial property for synthetic mathematics, as in Section 2.5, is the existence of interesting models of the theory. Ideally, we would like to be able to interpret these results in topological spaces or even any (Grothendieck) ∞ -topos. Currently, these questions have not been fully resolved for the various cubical type theories that have been considered. In fact, Sattler (2018) has shown that the standard model of Cubical Agda is *not* equivalent¹³ to topological spaces. However, if one drops the reversal operation (`~ _`) from Cubical Agda, any internal result about homotopy groups of spheres corresponds to a result about the homotopy groups of spheres in spaces.¹⁴ Furthermore, there has been recent progress on an “equivariant” cubical set model that is equivalent to spaces (Riehl, 2019). We are hence very optimistic that these issues will be resolved in the near future. Furthermore, as soon as a satisfactory cubical type theory with a model in spaces has been developed, we expect it to be straightforward to adapt Cubical Agda and its library to that theory. Indeed, the main features that we rely on—computational univalence and HITs with definitional computation rules for all constructors—should also be satisfied by that cubical type theory.

The syntax and semantics of HITs in cubical type theory were studied in Coquand *et al.* (2018). The canonicity proof has been shown to extend to the circle and propositional truncation in Huber (2016, Section 5). One technical consequence of the way the system

¹³ By “equivalent,” we mean that the notion of fibration in the cubical set model gives rise to a model structure that is Quillen equivalent to the classical Quillen model structure on spaces.

¹⁴ For further details and discussions about this result, see: <https://groups.google.com/forum/#!topic/homotopytypetheory/imPb56IqxOI>.

in Coquand *et al.* (2018) is designed is that there are closed terms of the circle in an empty context that are not `base`, for example, `hcomp (λ i → empty) base`. These degenerate elements were a serious problem in `cubicaltt` as they complicated both programming and proving, affecting the efficiency of the system.

These elements arose from the way `comp` reduces for `Glue` types in Cohen *et al.* (2018), but with the optimization discussed in Section 5.2 using `ghcomp` we can eliminate them. This requires us to impose a “validity” constraint on partial elements—following Angiuli *et al.* (2018, Definition 12)—which says that `Partial r A` is *valid* if it cannot become `empty` from a dimension substitution (a concrete condition is that r is a classical tautology). Validity combined with `ghcomp` eliminates all of the ways that a partial element can become `empty` in the system. As Cubical Agda implement the `ghcomp` optimization we expect it to be possible to prove a refinement of the canonicity theorem stating that the point constructors are the only elements of HITs in the empty context.

While the `comp` operation is complicated, a recent result by Coquand *et al.* (2019) shows that for the Cohen *et al.* (2018) cubical type theory any implementation of the `comp` operation yield the same result for natural numbers up to a path. As Cubical Agda is based on this cubical type theory the result also applies, so even though the implementation of `comp` differs from the way `comp` was defined in Cohen *et al.* (2018), the result for closed terms of type natural numbers will be the same up to a path.

7 Conclusion

In this paper, we presented Cubical Agda, an extension of Agda with features from cubical type theory. This brings to a proof assistant both a fully computational univalence principle and HITs. Moreover, induction on HITs and construction of paths are integrated into Agda’s very expressive pattern matching, providing support for more idiomatic definitions than direct use of eliminators. We expect that such a development environment will lead to more widespread use and experimentation not only of cubical type theory but also of HoTT/UF, in particular for programming applications.

7.1 Related work

This work is based on the work on cubical type theory of Cohen *et al.* (2018) and Coquand *et al.* (2018) and the `cubicaltt` prototype implementation (Cohen *et al.*, 2015). However, that implementation did not have support for many of the features of a modern proof assistant (implicit arguments, type inference, powerful pattern matching, etc.), so Cubical Agda can be seen as its successor. Additionally, the transport structure for inductive families is based on the schema presented in Cavallo & Harper (2019).

The most closely related cubical proof assistant to Cubical Agda is `redtt` (The RedPRL Development Team, 2018), which also supports computable univalence and HITs. It is based on a variation of cubical type theory called *cartesian* cubical type theory. This has models in cartesian cubical sets (Angiuli *et al.*, 2019) and cartesian cubical computational type theory (Angiuli *et al.*, 2018; Cavallo & Harper, 2019). The `redtt` system has been developed from scratch in order to be a proof assistant for cubical type theory and

it has some features that are not in Cubical Agda yet, like pretype universes and extension types inspired by Riehl & Shulman (2017).

The work of Tabareau *et al.* (2018, 2020) extends Coq with the ability to transport programs and properties along equivalences using what the authors call *univalent parametricity*. While this achieves some consequences of constructive univalence, it does not provide computational content to the full univalence axiom, in particular to neither function nor propositional extensionality. There is also no support for HITs.

The computation rules for equality are also defined by cases on the type in *Observational Type Theory* (OTT) (unpublished data; Altenkirch *et al.*, 2007). This type theory also proves function and propositional extensionality without sacrificing type checking and constructivity; however, it satisfies UIP. Recently, the XTT type theory has been developed (Sterling *et al.*, 2019) to reconstruct OTT's exact equality using cubical methods, satisfying UIP rather than univalence. Languages like XTT and OTT can be used as an extensional substrate for a two-level type theory (Voevodsky, 2013; Annenkov *et al.*, 2017), which would have both equality and path types.

Examples of ideas from HoTT/UF in computer science include Angiuli *et al.* (2016) where the authors use univalence and HITs to model Darc's style patch theory. This work envisioned what could be done if these notions were computing, but at the time it was unknown how to make this happen. However, now that Cubical Agda supports this, it would be interesting to redo the examples as the implementation would now compute. Another example is HoTTSQL (Chu *et al.*, 2017) which defines a formal SQL style language. The use of HoTT/UF is restricted to reasoning about cardinal numbers and it is not clear how much would be gained from doing this cubically.

Since the conference version (Vezzosi *et al.*, 2019) of this article was published, some interesting formalizations have been performed using Cubical Agda. Forsberg *et al.* (2020) implemented three equivalent ordinal notations systems and transported programs and proofs between them. Altenkirch & Scoccola (2020) considered a higher inductive version of the integers which differs from the one in Section 2.4.1. Veltri & Vezzosi (2020) formalize the π -calculus using a *guarded* version of Cubical Agda (Birkedal *et al.*, 2019). Various results from synthetic homotopy theory, extending Section 2.5, were developed by Mörtberg & Pujet (2020). Finally, Angiuli *et al.* (2020) explored the consequences of cubical type theory and Cubical Agda to traditional computer science applications like program/proof transfer and representation independence.

7.2 Future work

Interesting further directions would be to study meta-theoretical properties of cubical type theory, including a proof of decidability of type checking and a complete correctness proof of the conversion checking algorithm with respect to a declarative specification of equality. We believe this can be done by extending the canonicity proof of Huber (2016) using ideas from Abel *et al.* (2017).

We would also like to extend Cubical Agda with more cubical features, like cubical extension types inspired by Riehl & Shulman (2017). An important open problem in the area of constructive synthetic homotopy theory is to compute the Brunerie number (Brunerie, 2016) which so far has proved to be infeasible using cubicaltt and

Cubical Agda. It would hence be interesting to study compilation and efficient closed term evaluators of cubical languages in order to be able to do this kind of computations.

Acknowledgments. The authors are grateful to the anonymous reviewers for their helpful comments on earlier versions of the paper. The second author is also grateful for the feedback from everyone in the Agda learning group at CMU and especially to Loïc Pujet for porting the HIT integers from `cubicaltt` and Zesen Qian for the formalization of set quotients. We would also like to thank Martín Escardó for encouraging us to develop the `agda/cubical` library and everyone who has contributed to it since then. Andrea Vezzosi was supported by a research grant (13156) from VILLUM FONDEN, and by USAF, Airforce office for scientific research, award FA9550-16-1-0029. Anders Mörtberg was supported by the Swedish Research Council (SRC, Vetenskapsrådet) under grant no. 2019-04545. Andreas Abel acknowledges support by the SRC under grants 621-2014-4864 and 2019-04216.

Conflicts of Interest. None.

Supplementary materials

For supplementary material for this article, please visit doi.org/10.1017/S0956796821000034

References

- Abel, A., Öhman, J. & Vezzosi, A. (2017) Decidability of conversion for type theory in type theory. *Proc. ACM Program. Lang.* **2**(POPL), 23:1–23:29.
- Abel, A., Pientka, B., Thibodeau, D. & Setzer, A. (2013) Copatterns: Programming infinite structures by observations. *SIGPLAN Not.* **48**(1), 27–38.
- Agda Development Team. (2018) *Agda 2.5.4.2 Documentation*.
- Ahrens, B., Capriotti, P. & Spadotti, R. (2015) Non-wellfounded trees in homotopy type theory. CoRR abs/1504.02949.
- Altenkirch, T., McBride, C. & Swierstra, W. (2007) Observational equality, now! In PLPV’07: Proceedings of the 2007 Workshop on Programming Languages Meets Program Verification. ACM, pp. 57–68.
- Altenkirch, T. & Scoccola, L. (2020) The integers as a higher inductive type. In Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science. LICS’20. Association for Computing Machinery, pp. 67–73.
- Angiuli, C., Brunerie, G., Coquand, T., Hou (Favonia), K.-B., Harper, R. & Licata, D. R. (2019) *Syntax and Models of Cartesian Cubical Type Theory*. Preprint.
- Angiuli, C., Cavallo, E., Mörtberg, A. & Zeuner, M. (2020) *Internalizing Representation Independence with Univalence*. Preprint arXiv:2009.05547v2 [cs.PL].
- Angiuli, C., Hou (Favonia), K.-B. & Harper, R. (2017) *Computational Higher Type Theory III: Univalent Universes and Exact Equality*. Preprint arXiv:1712.01800v1.
- Angiuli, C., Hou (Favonia), K.-B. & Harper, R. (2018) Cartesian cubical computational type theory: Constructive reasoning with paths and equalities. In 27th EACSL Annual Conference on Computer Science Logic (CSL 2018), Ghica, D. & Jung, A. (eds), Leibniz International Proceedings in Informatics (LIPIcs), vol. 119. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, pp. 6:1–6:17.

- Angiuli, C., Morehouse, E., Licata, D. R. & Harper, R. (2016) Homotopical patch theory. *J. Funct. Program.* **26**, e18.
- Annenkov, D., Capriotti, P. & Kraus, N. (2017) *Two-Level Type Theory and Applications*.
- Bickford, M. (2018) Formalizing Category Theory and Presheaf Models of Type Theory in Nuprl. CoRR abs/1806.06114.
- Birkedal, L., Bizjak, A., Clouston, R., Grathwohl, H. B., Spitters, B. & Vezzosi, A. (2019) Guarded cubical type theory. *J. Auto. Reasoning* **63**(2), 211–253.
- Brady, E. (2013) Idris, a general-purpose dependently typed programming language: Design and implementation. *J. Funct. Program.* **23**(5), 552–593.
- Brunerie, G. (2016) *On the Homotopy Groups of Spheres in Homotopy Type Theory*. PhD thesis, Université de Nice.
- Cavallo, E. & Harper, R. (2019) Higher inductive types in cubical computational type theory. *Proc. ACM Program. Lang.* **3**(POPL), 1:1–1:27.
- Chu, S., Weitz, K., Cheung, A. & Suciu, D. (2017) Hottsql: Proving query rewrites with univalent sql semantics. *SIGPLAN Not.* **52**(6), 510–524.
- Cockx, J. & Abel, A. (2018) Elaborating dependent (co)pattern matching. *Proc. ACM Program. Lang.* **2**(ICFP), 75:1–75:30.
- Cockx, J. & Devriese, D. (2018) Proof-relevant unification: Dependent pattern matching with only the axioms of your type theory. *J. Funct. Program.* **28**, e12.
- Cohen, C., Coquand, T., Huber, S. & Mörtberg, A. (2015) *Cubicaltt*. <https://github.com/mortberg/cubicaltt>.
- Cohen, C., Coquand, T., Huber, S. & Mörtberg, A. (2018) Cubical type theory: A constructive interpretation of the univalence axiom. In *Types for Proofs and Programs (TYPES 2015)*. LIPIcs, vol. 69, pp. 5:1–5:34.
- Cohen, C., Dénès, M. & Mörtberg, A. (2013) Refinements for free! In *Certified Programs and Proofs*, Gonthier, G. & Norrish, M. (eds). Lecture Notes in Computer Science, vol. 8307. Springer International Publishing, pp. 147–162.
- Coquand, T. & Danielsson, N. A. (2013) Isomorphism is equality. *Indagationes Mathematicae* **24**(4), 1105–1120.
- Coquand, T., Huber, S. & Mörtberg, A. (2018) On higher inductive types in cubical type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science. LICS'18*. ACM, pp. 255–264.
- Coquand, T., Huber, S. & Sattler, C. (2019) *Homotopy Canonicity for Cubical Type Theory*. Preprint available at <http://www.cse.chalmers.se/~simonhu/papers/can.pdf>.
- Danielsson, N. A. (2020) *Higher Inductive Type Eliminator Without Paths*. <http://www.cse.chalmers.se/~nad/publications/danielsson-hits-without-paths.html>.
- de Moura, L., Kong, S., Avigad, J., van Doorn, F. & von Raumer, J. (2015) The lean theorem prover. In *Automated Deduction - CADE-25, 25th International Conference on Automated Deduction*, Berlin, Germany, August 1–7, 2015, Proceedings.
- Escardó, M. H. (2019) *Introduction to Univalent Foundations of Mathematics with Agda*.
- Forsberg, F. N., Xu, C. & Ghani, N. (2020) Three equivalent ordinal notation systems in cubical Agda. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. CPP 2020*. Association for Computing Machinery, pp. 172–185.
- Huber, S. (2016) *Canonicity for Cubical Type Theory*. Preprint arXiv:1607.04156.
- Huber, S. (2017) *A Cubical Type Theory for Higher Inductive Types*.
- Kapulkin, C. & Lumsdaine, P. L. (2012) *The Simplicial Model of Univalent Foundations (after Voevodsky)*. Preprint arXiv:1211.2851v4.
- Licata, D. R. & Brunerie, G. (2015) A cubical approach to synthetic homotopy theory. In *30th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS'15*. ACM, pp. 92–103.
- Licata, D. R., Orton, I., Pitts, A. M. & Spitters, B. (2018) Internal universes in models of homotopy type theory. In *3rd International Conference on Formal Structures for Computation and Deduction, FSCD 2018, July 9–12, 2018, Oxford, UK*, Kirchner, H. (ed). LIPIcs, vol. 108. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, pp. 22:1–22:17.

- Licata, D. R. & Shulman, M. (2013) Calculating the fundamental group of the circle in homotopy type theory. In Proceedings of the 2013 28th Annual ACM/IEEE Symposium on Logic in Computer Science. LICS'13, pp. 223–232.
- Lumsdaine, P. L. & Shulman, M. (2017) *Semantics of Higher Inductive Types*. Preprint arXiv:1705.07088.
- Martin-Löf, P. (1975) An intuitionistic theory of types: Predicative part. In *Logic Colloquium '73*, Rose, H. E. & Shepherdson, J. (eds). Amsterdam: North-Holland, pp. 73–118.
- McBride, C. (2009) Let's see how things unfold: Reconciling the infinite with the intensional. In Proceedings of the 3rd International Conference on Algebra and Coalgebra in Computer Science. CALCO'09. Springer-Verlag, pp. 113–126.
- Mörtberg, A. & Pujet, L. (2020) Cubical synthetic homotopy theory. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. CPP 2020. Association for Computing Machinery, pp. 158–171.
- Orton, I. & Pitts, A. M. (2016) Axioms for modelling cubical type theory in a topos. In 25th EACSL Annual Conference on Computer Science Logic (CSL 2016). LIPIcs 62, pp. 24:1–24:19.
- Riehl, E. (2019) The equivariant uniform Kan fibration model of cubical homotopy type theory. Talk given at The International Conference on Homotopy Type Theory (HoTT 2019) at Carnegie Mellon University.
- Riehl, E. & Shulman, M. (2017) A type theory for synthetic ∞ -categories. *Higher Struct.* 1(1), 147–224.
- Sattler, C. (2018) Do cubical models of type theory also model homotopy types? Talk given at Types, Homotopy Type Theory, and Verification at the Hausdorff Center for Mathematics in Bonn.
- Sojakova, K. (2016) The equivalence of the torus and the product of two circles in homotopy type theory. *ACM Trans. Comput. Logic* 17(4), 29:1–29:19.
- Sterling, J., Angiuli, C. & Gratzer, D. (2019) Cubical syntax for reflection-free extensional equality. In 4th International Conference on Formal Structures for Computation and Deduction, FSCD 2019. Leibniz International Proceedings in Informatics, LIPIcs, Geuvers, H. (ed). Schloss Dagstuhl- Leibniz-Zentrum für Informatik GmbH, Dagstuhl Publishing.
- Tabareau, N., Tanter, E. & Sozeau, M. (2018) Equivalences for free: Univalent parametricity for effective transport. *Proc. ACM Program. Lang.* 2(ICFP), 92:1–92:29.
- Tabareau, N., Tanter, É. & Sozeau, M. (2020) *The Marriage of Univalence and Parametricity*. Preprint arXiv:1909.05027 [cs.PL].
- Team, T. C. D. (2019) *The Coq Proof Assistant, version 8.9.0*.
- The RedPRL Development Team. (2018) *The redtt Proof Assistant*.
- Univalent Foundations Program, T. (2013) *Homotopy Type Theory: Univalent Foundations of Mathematics*.
- Veltri, N. & Vezzosi, A. (2020) Formalizing π -calculus in guarded cubical Agda. In Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. CPP 2020. Association for Computing Machinery, pp. 270–283.
- Vezzosi, A. (2017) *Streams for Cubical Type Theory*. <http://saizan.github.io/streams-ctt.pdf>.
- Vezzosi, A., Mörtberg, A. & Abel, A. (2019) Cubical Agda: A dependently typed programming language with univalence and higher inductive types. *Proc. ACM Program. Lang.* 3(ICFP), 87:1–87:29.
- Voevodsky, V. (2013) *A Simple Type System with Two Identity Types*. <https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/HTS.pdf>.
- Voevodsky, V. (2015) An experimental library of formalized mathematics based on the univalent foundations. *Math. Struct. Comput. Sci.* 25, 1278–1294.
- Wood, J. (2019) *Vectors and Matrices in Agda*. Blog post at <https://personal.cis.strath.ac.uk/james.wood.100/blog/html/VecMat.html>.